Master's thesis

Master's Programme in Computer Science

Game engine implementation in the Rust programming language

Jens Pitkänen

June 24, 2025

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Jens Pitkänen

Työn nimi — Arbetets titel — Title

Game engine implementation in the Rust programming language

Ohjaajat — Handledare — Supervisors

Prof. S. Puglisi, Dr. J. N. Alanko

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	June 24, 2025	53 pages, 2 appendix pages

Tiivistelmä — Referat — Abstract

Game engines are the core software behind video games, with a focus on performance and handling low-level interactions with the operating system. Given those requirements, it is understandable that C++ is a popular choice of programming language for implementing them. Rust is recent programming language targeting similar use cases to C++, providing comparable performance and low-level capabilities, with a novel approach to memory safety via compile-time validation, rather than runtime safety measures like garbage collection. In this thesis, Rust's suitability to the particular use case of game engine development was studied using the Design Science Research Methodology. A prototype game engine was implemented using Rust as the programming language. The design of the prototype was based on existing literature about game engine architecture, existing Rust game engines, and the author's own experiences in hobbyist game development. The development process was documented for later analysis by taking notes and tracking time spent on development tasks. Rust was found to be a viable programming language for game engine development, at least for small games. The memory safety features of Rust were noted as helpful, even with a large amount of unsafe code which required manual review and testing with the Miri interpreter to validate. In particular, even though the allocators and multithreading were implemented as part of the game engine rather than using the standard library, many memory safety related benefits were still observed. Rust's borrows with compile-time validated lifetimes were found to be a good fit for a large part of the dynamic allocations made by the engine, and the safety features around multithreading were found to be useful for implementing a convenient and safe parallelization interface.

ACM Computing Classification System (CCS)
Software and its engineering → Software creation and management
→ Designing software → Software implementation planning → **Software design techniques**
Software and its engineering → Software notations and tools
→ General programming languages → **Language features**
Software and its engineering → Software organization and properties
→ Software system structures → **Real-time systems software**

Avainsanat — Nyckelord — Keywords

Rust, game engine, game development

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

1 Introduction **1**

2 Related work **3**

2.1 Game engines 3

2.2 The Rust programming language 4

2.3 Game engine internals 6

2.4 Existing Rust game engines 11

2.4.1 Bevy 12

2.4.2 Ggez 13

3 Methods **15**

3.1 Problem and motivation 15

3.2 Objectives of a solution 16

3.3 Design and Development 17

3.4 Demonstration 17

3.5 Evaluation 18

4 Design and development **19**

4.1 Design 19

4.2 Development process 26

5 Results **36**

5.1 Observations from the development process 36

5.2 Time tracking 38

5.3 Demonstration 40

5.4 Evaluation 44

6 Discussion **46**

6.1 Research Questions 46

6.2 Limitations 47
6.3 Conclusion 48

Bibliography **51**

A Development time table

1 Introduction

Game engines are the complex software underlying video games, made by large game development companies to be shared by all of their game projects, or in some cases, to be licensed out to other game developers that need one. Since video games are soft real-time systems, that have to process everything needed to produce a frame of graphics and simulate the gameplay logic to produce those graphics in just a few tens of milliseconds, if not fewer, their development and design revolves around maintaining high performance and low latency within these complex codebases.

Due to the strict technical requirements, game engines are traditionally written in C++, a programming language that enables writing highly performant code, as well as easy access to low-level interfaces like operating system APIs, to avoid any unnecessary overhead and maximum control of the performance and memory usage of the software written with it.

In the last decade, the Rust programming language, with similar capabilities to C++, which differs in particular from C++ by being a memory safe language, has become increasingly popular within other fields with similar requirements: databases, networking infrastructure, and most recently, operating systems themselves. Though being usable for similar use cases, Rust features a different paradigm to the C++'s object-oriented programming paradigm, having some features reminiscent of functional programming languages, as well as having a distinctive way of validating references with the borrow checker. The concepts of lifetimes and ownership of objects, validated at compile-time, make the design of Rust programs quite different from C++.

This thesis explores how well the Rust programming language fits the use case of game engine development, using the Design Science Research Methodology. As a source data, a prototype game engine was designed and developed, tracking the process for both qualitative data in the form of observations of the process, as well as quantitative data in the form of tracking how time was spent on various topics that were identified as possible time sinks when implementing a game engine in Rust. Additionally, a short game was developed for the Ludum Dare 57 game jam, to evaluate how well the prototype fulfilled its purpose as a game engine.

To provide a focus to structure this thesis studying Rust in game engine development around, the following research questions are set out to be answered in this thesis:

RQ1: Which parts of game engine development are hindered by using Rust?

RQ2: Which game engine design patterns fit Rust well?

RQ3: How do Rust's language features affect game engine API design?

This thesis begins with an overview of game engines and the Rust programming language in Chapter 2. In Chapter 3, the Design Science Research Methodology is explained, how this thesis fits into the framework, and how the design and development process under study was planned. The design and development process is described in detail in Chapter 4, which is then distilled into Chapter 5, where the most relevant details of the process are presented. Chapter 6 discusses the results from the perspective of the research questions set out above, explores the limitations of the findings of this thesis, and concludes the thesis, presenting possible avenues for future research around this topic.

2 Related work

This chapter reviews existing literature on game engines and the Rust programming language, as well as existing game engines written in Rust. The first section defines what a game engine is and motivates this research, followed by a description of the Rust programming language, as well as its applicability as an implementation language for game engines. The third section is a general overview of what game engines are and what systems they are made of, and in the final section, existing game engines implemented in Rust are examined, to see what existing solutions look like.

2.1 Game engines

In their paper about game engine terminology [23], Toftedahl and Engström propose a taxonomy for tools used to make games, based on which they define a *game engine* as: "a piece of software that contains a core engine and an arbitrary number of user facing tools." In this thesis, the user facing tools are not considered for the most part. However, the *core engine* mentioned in the definition is in turn defined as: "a collection of product facing tools used to compile games to be executed on target platforms." In this thesis, these core engines are the main focus. The *product facing tools* in the definition of a core engine refer to the software that ends up in the final product, for example the graphics rendering, physics simulation, and artificial intelligence systems. This limits the scope of this thesis to a more specific area of game engine development, which is more focused on performance and optimization — topics which are affected by programming language choice. Since they are the focus of this thesis, the term *game engine* will be used to refer to core engines specifically.

Interest into game engine research is shown in a paper by E. F. Anderson et al. [2], calling for research into various aspects of game engines. Specifically, in Section 2.4 of the paper, they ask: "Design Dependencies: How do low-level issues affect top-level design?" The authors describe how various low-level changes can affect the top-level design of a game engine, and motivate research into what extent these changes may manifest, and if there are design patterns which would minimize the impact of such low-level changes on the top-level design. This shows an interest for academic research into game engines and their

development, especially the repercussions of low-level design specifics — in the case of this thesis, the implementation language.

As mentioned in *Game Engine Architecture, 3rd Edition* [9, p. 4], C++ is "the language of choice among most modern game developers," and the code examples given in the book are written in C++. This language choice makes a lot of sense for game engines: performance is key, and C++ allows writing high performance code without many restrictions. However, C++ suffers from memory safety issues, as noted in a blog post by the Microsoft Security Response Center [15], finding that around 70 % of relevant security vulnerabilities were caused by memory safety problems in C and C++ code. Even though games are not as concerned about security vulnerabilities as an operating system might be, it is evident that C++ code may behave differently than the programmer intended due to the lack of memory safety. In particular, game engines make use of manual memory management and multithreading, both of which are prone to memory safety issues by their nature. As such, a memory-safe language that provided the performance and low-level access game engines require, could help make game engines more stable and prevent bugs.

2.2 The Rust programming language

Rust is a programming language aimed at memory-safe systems programming applications. In their paper [7], Fulton et al. present interview and survey results from developers who were asked questions about using the Rust programming language. Among the participants, Rust was used in such performance and safety focused domains as databases, operating systems, and compilers. The participants listed Rust's safety and performance promises as the main reasons to use Rust, and found that Rust improves both confidence in the resulting software, as well as developer productivity. High performance is a concern shared with game engine developers, as it is an important feature for game engines: they need to do complex computation within a soft real-time environment, often simulating physics and rendering a more-or-less realistic image of a 3D virtual world every 16 milliseconds. The added confidence in the resulting software and developer productivity line up with game engine development needs as well, as game engines are large software projects that ship on various platforms, which benefit from all tools that can help maintain stability of the software while allowing development to progress. Static analyzers are a classic example of such a tool, and Carmack underlined the importance of static analysis in game engines in his blog post [4], noting that with a "large enough [C++] codebase,

any class of error that is syntactically legal probably exists there." It follows then, that Rust, a language with more guarantees about specific kinds of code correctness, could help in a similar manner.

Regarding performance, Zhang et al. studied the performance difference between Rust and C in various micro-benchmarks in their paper [26], and found Rust to take 1.77 times longer to execute on average, with the regular runtime safety checks enabled. However, disabling these checks improved Rust performance to only take 1.23 times longer, and further tweaking of the algorithms to avoid slowdown caused by safety requirements — the Rust standard library has less performant, but safer default behavior for strings and integers, enforcing strings to be valid UTF-8 and integers to have specific wrapping logic respectively — brought the performance to "indistinguishable" levels with C. This suggests that Rust, at the very least, would not outright prevent optimizing code to be as fast as it would be with a more traditional game engine implementation language such as C, though it is slower by default.

On the safety side, according to Qin et al. in their paper [17], Rust is "very effective in preventing memory bugs" as long as the code in question does not use Rust's `unsafe` marker to relax some of the language constraints for a block of code, and a stable version of the compiler is used. However, they did also find that unsafe code is sometimes unavoidable, so there are limits to the memory safety guarantees as a whole.

The clear trade-off to Rust's safety guarantees discussed above is that the code needs to satisfy more constraints, which makes programming with it more difficult, and the compiler needs to verify those constraints, which makes compilation slower. As studied by Zhu et al. in their paper [27], complex lifetimes and ownership issues cause friction while learning the language, with lifetimes being the harder topic to grasp. As a real-world example, LogLog Games published a blog post [14] about their personal opinion for why they stopped using Rust in their games, and they cite the borrow checker as a blocker for being productive in the language on multiple occasions. In particular, it is cited as a reason that Rust code needs to be refactored more often than other languages, harming iteration speed. Another issue mentioned in the blog post is long compile times, which have improved over time, but are still slow enough that they are among the most requested areas of improvement in the compiler, as can be seen from the annual Rust Survey [22].

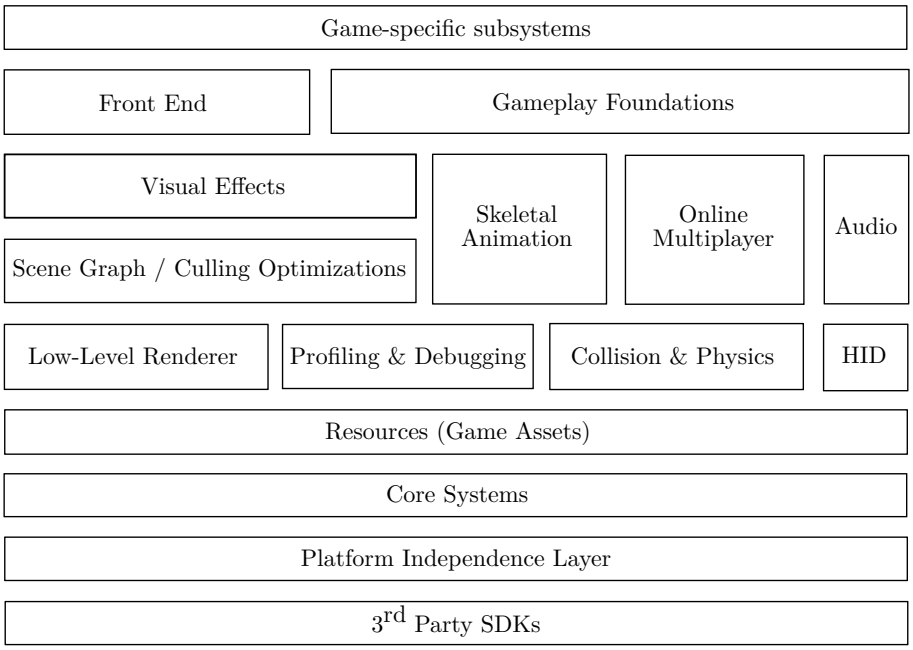

Figure 2.1: Diagram adapted from Figure 1.16 "Runtime game engine architecture" in *Game Engine Architecture, 3rd Edition* [9, p. 39] illustrating the parts a game engine is made of. Each block in the original diagram further elaborates on what subsystems it contains, this diagram only includes the titles for brevity. The original diagram also has the operating system, drivers, and hardware at the bottom of the diagram, but they are omitted for brevity as well.

2.3 Game engine internals

For a closer look at the systems a game engine contains, *Game Engine Architecture, 3rd Edition* by Gregory [9] describes the internals of game engines in great detail. As seen in Figure 2.1, a game engine — or a "runtime engine" in the book's terms — consists of many distinct systems which build on each other to form a game engine. The dependencies and references between these systems can be modeled in various ways, and this thesis will explore ways that are ergonomic to implement and use with the Rust programming language. The rest of this section will give an overview of each of the systems laid out in Figure 2.1, as described in the book.

3rd Party SDKs in Figure 2.1 refer to the often numerous libraries and tools integrated in game engines, licensed from a third party instead of being implemented by the engine developer themselves. Given the extensive integration these often require, they are placed at the base of the diagram, as the rest of the engine is shaped by them.

Platform Independence Layer refers to a wrapper around target platform specific APIs, such as file access, multithreading, and graphics processing unit (GPU) programming.

These wrappers are found in almost every engine, as the higher level systems can often be shared between platforms, and the wrappers allow easier deployment on multiple platforms. Game engines can specialize in specific target platforms such as mobile devices, but there can still be target platform differences to wrap — iOS and Android, in this example.

Multithreading is particularly relevant to this thesis, as one of Rust's major goals is "fearless concurrency," as mentioned in Chapter 16 of *The Rust Programming Language* [13], which ought to have an effect on how multithreading is implemented in a game engine. As described in *Game Engine Architecture, 3rd Edition* [9, pp. 544–558], there are three ways game engines tend to split up work between threads: one thread per subsystem, scatter/gather, and job systems. One thread per subsystem is self-explanatory and a relatively simple way to spread around work between CPU cores, but suffers from unbalanced core utilization, as some subsystems tend to require more processing time than others. Scatter/gather refers to a pattern where the main logic of the game engine is single-threaded, or possibly one thread per subsystem, but more specific work inside the subsystem is split to be done on multiple threads (scatter), and the results are combined after all the threads are done (gather). Finally, as the most complex pattern, but perhaps most efficient core utilization wise, job systems work by splitting the game engine processing work into smaller units called *jobs*, and a multithreaded scheduler then processes the jobs, spreading load between the cores. These patterns form a spectrum of trade-offs between ease of implementation and performance.

Core systems in Figure 2.1 covers the foundational parts of the engine: start-up, shutdown, how subsystems run and interact, as well as the tools required by all other systems, such as memory allocators.

Game engines have strict performance requirements, which can motivate implementing custom memory allocators for specific purposes [9, p. 427]. Stack-based allocators, also known as bump allocators or arenas, are noted as a popular option for their performance. Stack-based allocators allocate memory in a linear fashion, one allocation after another in memory, and disallow freeing individual allocations — the allocated memory can only be freed all at once. Pool allocators are also mentioned, as another performant type of allocator, for more longer lived objects. Pool allocators allocate objects of a fixed size, and let individual allocations be freed, which causes that "slot" to be added to a "free list," where future allocations can find appropriately sized available memory. Pool allocators are still faster than general purpose ones, with the tradeoff that they are usable only by objects of the same size.

Two specific use cases for stack-based allocators described in the book [9, pp. 434–437] are single-frame and double-buffered allocators: a single-frame allocator is reset at the start of each frame, while a double-buffered one switches between two single-frame allocators, delaying the reset by one frame, so that the previous frame's memory is always available. The advantage to double-buffering is that it allows persisting memory over time by "ping-ponging" data between the allocators, but it does also double the memory usage. While these frame allocators are great for performance, Gregory notes that a major issue is the discipline required from the programmers: pointers to a previous frame's memory — or pointers to more than one frame old data, for double-buffered allocators — must not be stored, to avoid use-after-free memory hazards.

Resources (Game Assets) in Figure 2.1 refer to the resource management systems included in game engines. Resources in this context refer to game assets such as images, 3D models, audio files, game levels, and everything else a game is made of, except for the code. These files are usually authored in a separate program, such as a 3D modeling application, a digital audio workstation (DAW), or another program for the other kinds of resources, and then converted into a format usable by these resource management systems via the *asset conditioning pipeline* (ACP), as described in *Game Engine Architecture, 3rd Edition* [9, pp. 61–64]. The ACP is usually considered a part of the game engine, but it is separate in the sense that it is a separate program, ran during development ("offline"), and once the game is being played ("online") the files have already been processed through this pipeline. The ACP usually consists of a step where the source file, used by the authoring program, is exported into some interchange format, and then converted into some format the game engine itself understands. As an example, a Photoshop PSD file could be the source file, which would then get exported to PNG as the interchange format, which in turn would be processed by the game engine's tooling from PNG into a GPU-readable format which is understood by the rest of the engine. The GPU-readable pixels and relevant metadata is then inserted into a *resource database*, or simply saved as a file in the file system, somewhere where the game can find it.

After the resources have been converted into formats usable by the game engine, they still need to be read off the disk, either during a loading screen, or asynchronously streamed during gameplay, and possibly post-processed after that. This *runtime resource management system*, as described in Chapter 7 of *Game Engine Architecture, 3rd Edition* [9, pp. 481–523], handles finding the right files the game needs based on some kind of identifier the game code can pass in, a file path or a GUID. It also manages the memory needed

by the resources, which parts of the files are relevant in a particular situation, and when they should be loaded and unloaded. Loading a music track in parts as it is playing can save some memory, compared to loading many minutes of raw audio data to sit in RAM. Some assets, like textures and meshes, need another loading step after file reading as well, where GPU memory is allocated, and the data is moved over. In loading and unloading them, the resource manager manages lifetimes of the resources, which has effects on which memory allocation strategies can be used: simple lifetimes like loading all the data for one specific game level at a time, and unloading that data all at once, allow for simple memory allocation schemes such as stack-allocation. Complex streaming systems on the other hand require careful memory management, such as splitting all the assets into fixed size chunks to make swapping them out easier, and reference counting the resources to see what can be unloaded.

HID, or Human Interface Devices, in Figure 2.1 refer to the system that detects and tracks the devices that the player uses to interface with games. Games also make use of a few patterns of input which this system should be able to provide via a higher level API: detecting chords of multiple buttons being pressed at once, sequences of different buttons, gestures with analog inputs, and so on.

Profiling and Debugging in Figure 2.1 make up systems that are not used as much in the distributed game, but are vital for efficient game development processes: logging, tracing for performance profiling, debug drawing capabilities for visualizing normally invisible relations between objects or object states, crash reporting, developer configuration menus, cheats to speed up testing, and memory tracking systems.

Graphics related subsystems in Figure 2.1 are the *Low-Level Renderer*, *Scene Graph*, *Visual Effects*, and *Front End*. All these contribute to rendering the visual portion of the game. These systems are often at the center of game engine design, as the renderer is an especially performance-sensitive subsystem, requiring strict coordination between low-level systems for managing GPU memory and asynchronous processes, with the high-level systems deciding what to draw in the first place. The *scene graph* stores the objects to render, and filters out objects that can be deemed to not appear on screen. The resulting objects are then passed to the *low-level renderer*, which renders those objects on the GPU. *Visual effects* cover additional rendering systems on top of this, such as post-processing effects applied after the scene has been initially rendered, and the *front end* refers to 2D user interface elements which are rendered on top of everything else.

Online Multiplayer in Figure 2.1 refers to systems which manage online session creation

and game state synchronization between players across the network. While this online functionality is not relevant to many games, it is relevant to consider early in the design of an engine, as explained in *Game Engine Architecture, 3rd Edition* [9, p. 55]. Game engines designed for single-player are notoriously hard to convert to multiplayer, since it requires finding every part of the game that needs synchronization, and synchronizing that data over the network. On the other hand, using a game engine designed for multiplayer can work very well for single-player games, as a single-player game can be thought of as a multiplayer game with only one player.

Skeletal Animation in Figure 2.1 refers to the system that handles skeletal animation, a common method of animating characters and other graphical elements in games using a set of transform matrices, often referred to as "bones," which can be used to bend 3D models around by applying those transforms to different vertices of the models based on per-vertex weights. Skeletal animation often also interacts with the physics and gameplay systems to achieve more a more realistic look, which makes the skeletal animation subsystem have wide ranging effects on the rest of the game engine. In particular, animation data structures often need to refer to other data related to the object being animated, and in turn, other systems related to that object may need to know of its current animation state. These requirements make the skeletal animation system especially interconnected with other systems within the engine.

Collision and Physics in Figure 2.1 are made up of systems to detect collisions between objects in the game, and sometimes a system for simulating realistic looking physics for those objects — not all game engines implement physics simulation, but practically all do have some sort of collision detection. Collision detection, and especially physics simulation, requires a lot of processing, and can take a large part of the processing power required by a game alongside rendering. Perhaps due to the difficulty of implementation or optimization of this system, many game engines use a third party physics SDK which is kept synchronized with the rest of the engine, and queried for "physics state" as needed.

Audio in Figure 2.1 covers the systems which manage audio devices to play back audio from, mix different audio sources together, and apply effects on that audio. The audio system can be particularly connected to the resource management system, as even in otherwise simple resource managers, streaming long audio files can have outsize benefits since only a small part of the whole audio is ever needed at a time.

Gameplay foundations in Figure 2.1 contains the systems where the line between game and game engine starts to blur. Still, there are some high-level systems which are performance

sensitive and technically complicated to implement, which makes them better suited to being implemented inside the engine, such as managing how the virtual objects in the game world are represented in memory and code, scripting, and event systems. One particularly relevant architectural decision regarding these systems is discussed in section 16.2 of *Game Engine Architecture, 3rd Edition* [9, p. 1043] about *runtime object model architectures*. The first architecture presented is *object-centric*, which is perhaps the more intuitive one for object-oriented languages like C++: *runtime objects* — objects in the virtual game world the player can observe directly or indirectly while playing — are represented by instances of classes which are implemented to handle everything related to how the object appears and behaves. For code reuse, this kind of architecture often involves class hierarchies, where the ancestor classes can describe shared features, such as rendering and movement logic. The book also explains approach to code reuse with an object-centric design: a component based architecture, where the objects act as containers for instances of "component" classes, and those components implement the actual logic.

The other approach to runtime object model architectures described in *Game Engine Architecture, 3rd Edition* [9, p. 1057] is *property-centric*. As the name suggests, instead of building the system from the perspective of the runtime objects, in a property-centric model, the properties of the runtime objects are central. Properties represent some reusable part of a runtime object, and different instances of the property are stored and processed together, rather than being spread out in the objects that contain them. Each runtime object made up of a set of these properties, generally via some kind of handle that can be used to find the specific runtime object's properties from their separate storages.

The property-centric design is particularly notable as a part of the *Entity-Component-System* (ECS) pattern, found in game engines such as Bevy [1], which is analyzed in more detail in Section 2.4. The *entities* match the above definition of runtime objects, the *components* match the definition of properties, and the *systems* are processes that operate on specific sets of properties. For example, a rendering system might access the "position" and "image" properties of each runtime object that has those properties, to render the applicable runtime objects' images in their intended positions.

2.4 Existing Rust game engines

While there is a lack of peer-reviewed literature regarding game engines written in Rust, they do exist. In this section, the most popular Rust game engines are reviewed for

insight into how they make use of Rust in particular, and how they might differ from the description of C++ game engines in Section 2.3.

To pick the engines to review in this section, a list was compiled from https://crates.io/ — the default host for Rust libraries — under the "Game engines" category, sorted by their Recent Downloads statistic as of September 2nd, 2024. The category also includes smaller libraries that are intended to be used as part of game engines, but those are ignored for the purposes of this list. The top four engines found were Bevy [1], ggez [8], Amethyst [12], and Tetra [5]. Of these, Amethyst development seems to have stopped completely, with the public repository being read-only, and Tetra is "passively maintained," so this section will focus on Bevy and ggez.

2.4.1 Bevy

Bevy [1] is a game engine written in Rust which leans heavily into the ECS architecture. The engine is modular, allowing developers to pick and choose the systems they need: the various features provided by game engines described in Section 2.3 are exposed as distinct *systems* and *components* in ECS terms.

Both the engine and game logic in Bevy is written in functions representing *systems*, which are called by a scheduler as the game runs. These systems access shared and persistent state by defining *queries* and *resources* as their function parameters, and the ECS back end handles passing in the query results and resources to the system functions. Both queries and resources are represented by generic types whose type parameters are used to define what state the system wants to access. Queries are used to iterate through all the entities that have a specific set of components. Where many entities can have different instances of a component, resources are one-of-a-kind: only one instance of a specific resource exists at a time, and that instance is passed to each system that has the resource as a parameter.

Given how central components are to the design, it is safe to say that Bevy has a property-centric runtime object model architecture. Bevy handles the problem of finding the correct property instances for each relevant entity by making use of Rust generics, which allows a very ergonomic API on the system side — a system that needs entities with "position" and "image" components can define a function parameter with the type `Query<(&Position, &Image)>` and use it as an iterator to access those components for all the applicable entities.

The asset management system in Bevy is exposed as a "resource" in Bevy terms, so any system that requires some asset can define a parameter with the type `Res<AssetServer>`

to be able to load in new assets, which are represented by reference counted handles.

The system functions in Bevy are called by a scheduler, which spreads the computation load between CPU cores. This makes Bevy games multithreaded by default, though as pointed out by LogLog Games in their blog post [14], game logic can be quite strictly ordered, and ordering systems does prevent them from running on multiple threads. Bevy's systems run from start to end without interruption, and cannot spawn or start other systems to process some of the work in parallel. This model is similar to the thread pool based job system described in *Game Engine Architecture, 3rd Edition* [9, pp. 551–552], which is relatively simple and easy to reason about, but it can limit the scheduler's ability to spread the work between cores. It also doesn't include more sophisticated async/await-style communication between parallel jobs, as seen in more complex, fiber or coroutine based job systems.

Regarding the core systems, Bevy does not expose any custom allocators, but it does handle subsystem initialization and access differently from the C++ examples given in *Game Engine Architecture, 3rd Edition* [9, pp. 418–426]. In the book, subsystems are globally accessible, each subsystem's state stored in a static variable which can be used to initialize, call, and shut down those subsystems. In Rust, static variables are intentionally hard to use — memory safety is hard to guarantee for variables which can be accessed at any time, from any thread — so Bevy does not expose subsystems via static variables. Instead, these subsystems are ran in the Bevy ECS as systems, and the game logic interacts with them via creating, mutating, and removing specific components from entities. For example, an animation can be played by adding an `AnimationTransitions` component to an entity with an animatable mesh, and calling its `play` function to mutate the component to signal that an animation should be played. This causes the animation system implemented in Bevy to start playing the animation when it gets called by the scheduler. Calling any relevant startup and cleanup functions of the subsystems is handled by the scheduler as well.

2.4.2 Ggez

Ggez [8] is an older Rust game engine compared to Bevy, and as such, might have less benefit from hindsight in its design. However, it is also built with a fundamentally different architecture from Bevy, as it is not exactly an entire game engine, but rather provides the lower level systems described in Figure 2.1. As mentioned in its README file, ggez does

not have physics simulation, animation, GUI, networking, or asset management systems. Notably, ggez does not include an ECS like Bevy, nor any other runtime object model.

Much like Bevy's ECS, ggez has a `Context` object that holds all the game engine state. On the game side, the `EventHandler` trait is implemented as an entry point for the game code, where the functionality of ggez can be used via the `Context` object. Where Bevy handles communication with the engine subsystems via creating and modifying components, ggez exposes them directly, via the fields of the `Context` structure, such as `gfx` for the graphics subsystem, and `audio` for the audio subsystem.

Ggez does not contain any specific utilities for multithreading, and while other Rust libraries may be used for this purpose, ggez might not integrate with them without issue. This is caused by many of the context objects being marked `!Send` and `!Sync`, which prevents accessing them from any other thread than the one they were created on.

While ggez is restricted to one thread, its subsystems are accessible in a more direct way than Bevy, and the features of the game engine as a whole seem relatively easy to use — access the subsystem via the `Context` object and call a function. This simplicity is an intentionally designed part of ggez, as it aims to have a similar developer experience to the LÖVE game engine [18] for making games in Lua with a similar straightforward API, but this evidently comes with some restrictions regarding performance, at least when it comes to multithreading.

3 Methods

This thesis uses the design science research methodology (DSRM), presented in Figure 3.1 and further explained in the article by Peffers et al. [16]. The methodology is designed for Design Science research in the field of Information Systems, which according to the paper is about "IT artifacts intended to solve identified organizational problems." Though the problem being solved is not organizational in nature, the methodology described in the paper is a good fit to answer the research questions set out in this thesis as well. The following sections describe in further detail how this thesis implements the DSRM, except for the final *Communication* step, which will be covered by the Master's thesis presentation and publication.

3.1 Problem and motivation

Game engines are complex and performance-sensitive software. Due to the performance-sensitivity, they are usually implemented in C++, which is not a memory-safe language. Memory safety issues can cause crashes and program state corruption, so they require some kind of prevention during development. Languages which are not memory-safe require careful programming to avoid memory safety issues, which increases development costs, and discourages patterns which have a higher probability of causing memory safety issues.

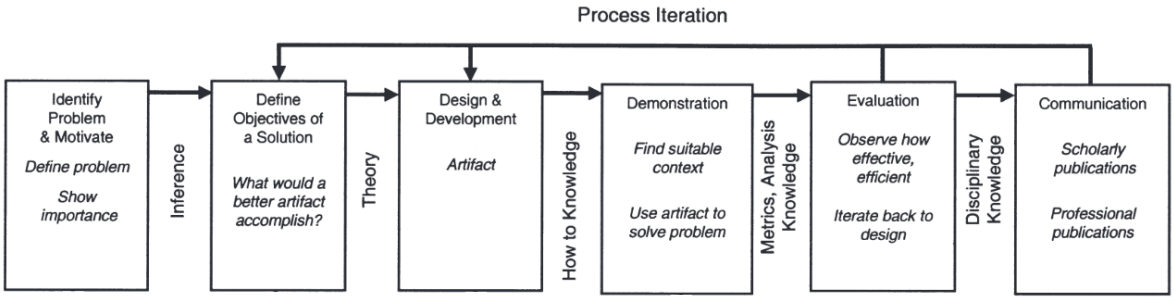

Figure 3.1: Diagram from Peffers et al. [16] outlining the research methodology used in the thesis.

3.2 Objectives of a solution

As seen in Chapter 2, Rust is a memory-safe language, with similar performance characteristics to C++, so it seems like an appropriate candidate for a game engine implementation language which would avoid the memory safety problem. As such, the solution explored in this thesis is using Rust as a game engine implementation language.

The artifact designed and developed for this thesis is a small prototype game engine, which has a few specifically picked features and attributes which should interact with the memory safety guarantees of Rust, as well as the restrictions imposed by Rust which are not seen in C++ engines. This approach should highlight both how memory safety can ease the process of developing a game engine, as well as the drawbacks caused by the restrictions from using memory-safe language. The objectives of the artifact are expressed as feature requirements for the game engine prototype:

O1: A custom memory allocator for per-frame data, on both the engine and game sides.

O2: A custom memory allocator for game assets, for the engine's resource manager.

O3: Memory-safe utilities for multithreading, for both the engine and game code.

O4: Engine systems should be easily usable from other systems as well as game code.

These objectives help explore and answer the research questions set out in Chapter 1 by stating more technically specific goals, which can then be used to answer the more general research questions.

As seen in Chapter 2, game engines can benefit from use-case specific memory allocators, which motivates objectives O1 and O2. Traditionally, these come with the caveat that those custom memory allocators need to be used carefully by programmers due to possible memory safety issues, so including them in the prototype should show the quality of the solution regarding the memory safety problem.

Multithreading is another important feature of game engines, as seen in Chapter 2, but memory safety issues can crop up when different threads access the same memory. As such, it makes up another feature that should underline the quality of the solution regarding the memory safety problem, which motivates objective O3.

Objective O4 is motivated from a different direction, as it should highlight the drawbacks of the solution rather than introducing memory safety issues to solve. As seen in Chapter 2,

existing Rust engines have varying solutions to accessing the various systems a game engine is made up of, each with their own performance and usability trade-offs. As this is a relatively simple issue for most game engines, with static variables and global access being ergonomic in C++, this should highlight some drawbacks of Rust's strict memory safety rules.

3.3 Design and Development

The artifact was designed to implement the smallest subset of the parts described in Section 2.3 which would still be usable as a game engine, and fulfill the objectives set out in Section 3.2. As game engines are large projects, the focus was on implementing a viable artifact with the least amount of work — in the end, the development process took about 240 hours over 4 months, from the first commit on November 25th, 2024, to the final changes made during the demonstration project on April 8th, 2025.

During the development process, challenges and progress were tracked on a task-by-task basis, resulting in a table of time spent on various parts of the artifact, alongside information about which of the expected challenges with Rust development actually materialized. This timetable is included in Appendix A, and further discussed in Section 5.2.

To track the developer experience from a qualitative perspective during design and development, informal notes were kept alongside the time tracking data, which were distilled into Chapter 4. Particularly salient insights from the development process are listed out in Section 5.1.

3.4 Demonstration

The artifact was used in a real-world scenario after its development to ensure its applicability to actual game development: a short game was made for the Ludum Dare 57 game jam, which used the artifact as its game engine. The game was finished within the 72 hour time limit, and included pathfinding, procedural map generation, character behavioral simulation, a user interface, tile-based graphics, and a few tracks of music. Based on this result, the artifact did fulfill its purpose as a game engine, at least for a game of this scale. The demonstration is further discussed in Section 5.3.

3.5 Evaluation

After the development and demonstration of the artifact, it was evaluated against the objectives previously set for it in this chapter. The artifact was found to fulfill all of the objectives set for it to at least some degree, with the final Objective O4 being the most questionable. Specifically, the architecture of the engine did work in the demonstration as well as within the artifact itself, but some concerns were raised about the generalizability of the design to larger projects. The evaluation of each objective is further expanded on in Section 5.4. After comparison against the objectives, the artifact's design and development process was discussed from the perspective of the research questions in Chapter 6.

4 Design and development

This chapter describes the development process of the game engine prototype, first from the design perspective and what design decisions were made, and then from the development perspective, focusing more on the actual development challenges, observations made during development, and implementation details relevant to them.

4.1 Design

As described in Section 2.3, game engines often implement their own platform abstractions and custom memory allocators, which this prototype was designed to do as well. Since the Rust standard library contains many abstractions for platform-specific functionality, and utilities that dynamically allocate memory, this prototype game engine was specifically designed around the *core* module of the Rust standard library. The core module does not depend on any system libraries, and thus does not implement those platform abstractions and dynamic allocation utilities [21]. Rust code can be restricted to only use the core module via the `#![no_std]` annotation, which was used in the runtime parts of the engine. This mode seems to fit game engines well, due to this preference for implementing a platform independence layer and allocators as part of the engine, instead of relying on standard library functionality.

Another priority while designing the engine was to keep its compile time low. Tooling, such as any tools involved in the asset conditioning pipeline's offline part, wasn't included in this, as they are normally not compiled alongside the game. The core engine which games depend on however, was designed around this idea, which manifested in two main rules: no procedural macros, and minimal use of generics. Procedural macros slow down compilation in two ways: they often depend on the syn library, which is a common large dependency used by procedural macros, and the procedural macros themselves are very powerful, which can cause unexpected slowdowns to the compilation process by generating a lot of "invisible" code. Generics cause slowdown due to the compiler generating code for each concrete type used in place of the generic, in a process called *monomorphization*, which slows down the compilation process, as described by Wood in his master's thesis about improving the Rust compiler's performance by eliminating unnecessarily duplicated

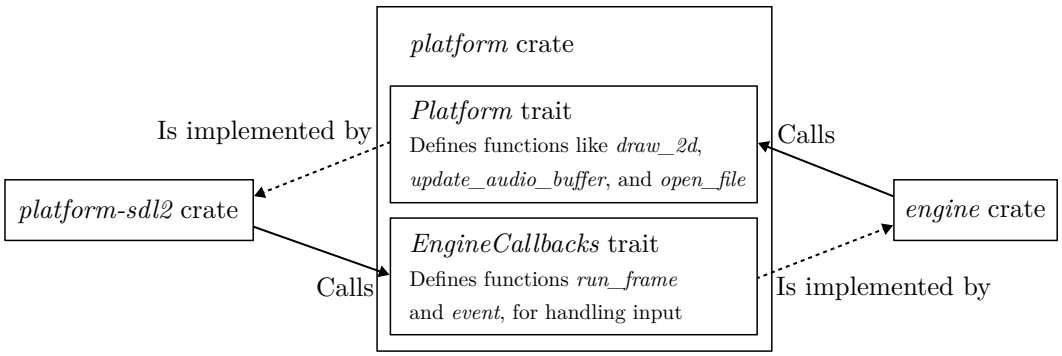

Figure 4.1: Dependencies between crates are shown as solid lines, with the arrow pointing towards the dependency. The dashed lines highlight how the platform implementation and engine can communicate without depending on each other — instead having the shared `platform` dependency — due to calling each other via `&dyn Trait` references, allowing them to be compiled in parallel.

code due to monomorphization [25]. Due to their performance impact, generics were only used where they provide a large benefit to ergonomics: container types, such a lists containing elements of a specific type. It should be noted that avoiding generics did not mean avoiding traits, as they do not have such a big effect on compile times when used as trait objects instead of generics. This did restrict their use to functions which are not called often, to avoid performance issues due to dynamic dispatch.

The focus of the prototype, when it comes to included features, was on the lower level parts of Figure 2.1. The higher level gameplay utilities were considered the easiest to implement on the game side as needed, so to limit the amount of development work needed for a usable prototype, the engine was designed to provide the core systems, a resource database, relatively low-level rendering and audio playback systems, and a HID abstraction.

The platform independence layer was designed to be highly decoupled from the rest of the engine. The `Platform` trait was designed for a separate platform implementation library to implement, while the platform-independent part of the engine would use it as a trait object. As seen in Figure 4.1, this made it possible for the engine and the platform implementation to communicate entirely via dynamic dispatch, which allowed them to be compiled in parallel, independent of each other. While the main engine contains much more code, the platform implementation often depends on some platform-specific libraries that can take a while to compile, so these two parts being independent avoids a costly serial dependency chain in the compilation process.

Multithreading was one of the more complex low-level systems designed for this prototype,

```
let context = get_context();
let src: [i32; 1024] = some_data();
let mut dst: [i32; 1024] = [0; 1024];
// ‘parallelize‘ splits ‘dst‘ into many subslices, each handled on a different thread.
parallelize(&mut engine.thread_pool, &mut dst, |subslice, offset| {
  // Each ‘subslice‘ is mutable, so each thread can mutate their own part of ‘dst‘.
  for (i, dst_element) in subslice.iter_mut().enumerate() {
    // The ‘offset‘ parameter can be used to map indices of this subslice to indices of
    // the entire slice. For example, to read from another slice with the same layout.
    let src_element = src[offset + i];
    // ‘context‘ and ‘src‘ can be accessed by all threads, since they are immutably borrowed.
    *dst_element = process(src_element, &context);
  }
});
```

Figure 4.2: Example usage of the `parallelize` function for scatter-gather multithreading.

so the focus was on making it as simple as possible to implement. The interface in the platform abstraction revolves around a first-in first-out *task queue*, which provides tasks for a *thread pool* to execute in order. Since the task queue was designed so that tasks can only be started and finished in order, the scheduling code could use a queue with relatively straightforward synchronization. While the core module of the Rust standard library does not provide any runtime functionality for multithreading, it does provide the fundamental traits `Send` and `Sync`. These traits are *auto traits*, which means that structures that only contain fields whose types have these traits in turn, are automatically `Send` and/or `Sync` as well. This makes it easy to detect which types are not safe to share between threads, so they were used to limit the multithreading API to only allow including `Sync` types in tasks. This allows the Rust compiler to check that any memory accesses done on the thread pool are safe: only types that are safe to use from multiple threads — those that implement `Sync` — can be used within the closures that are used to define the tasks to run on the thread pool.

On the engine side, an additional multithreading layer was designed to make the task queue and thread pool easier to use. This layer consists of a `parallelize` function that implements the scatter/gather approach to game engine multithreading discussed in Section 2.3 safely, by carefully managing which data is mutable and which is immutable in the multithreaded code. As seen in Figure 4.2, the function takes a mutable slice of data which the caller wants to mutate from multiple threads in parallel, and a closure that can

```
error[E0596]: cannot borrow ‘context‘ as mutable, as it is a captured variable in a ‘Fn‘ closure
   --> example/src/main.rs:289:53
    |
289 |                 *dst_element = process(src_element, &mut context);
    |                                                     ^^^^^^^^^^^^ cannot borrow as mutable
```

Figure 4.3: This is an example of the kind of compilation error Rust would give if a parallelized operation attempted to access the same memory mutably from multiple threads — this error would come from the example code in Figure 4.2 if `context` were mutable, and was borrowed mutably in the parallelized closure.

operate on that slice. The scatter/gather pattern is implemented inside the `parallelize` function, by splitting the given mutable slice into as many parts as the thread pool has threads, and then creating tasks for each slice, where the given closure is called and the appropriate slice passed in. The closure can contain immutable references to data outside the closure, which makes them more useful, as additional parameters can be implicitly passed via the environment captured by the closure.

As seen in Figure 4.3, mutable captured references — which would allow mutating the same values from multiple threads at once, causing hard-to-catch bugs as well as being unsound in Rust — are caught by the compiler, as a closure that could capture mutable references would not implement the `Fn` trait, which is used as a trait bound on the closure's generic type parameter. This way, there is a clear mutable part — the mutable slice of data — and a clear immutable part — the references captured by the closure — which can be reasoned about and safely passed into the threaded function calls. In the case of the mutable data, it is split so that each thread gets its own distinct part to process, and in the case of immutable data, each thread gets the same references, as they only need to access the data immutably. Both references also have `Sync` as a trait bound, to ensure they are safe to share between threads in the first place.

Similarly, as another relatively complex part, the allocators for the engine were designed to be simple to implement, but useful enough to fit the needs of the rest of the engine. The main allocator passed around to almost all parts of the engine is a linear allocator that allocates out memory at subsequent offsets into a backing memory area, and does not provide any way to free individual allocations. It is fast and simple, and in case the lifetimes of the allocations are clearly delimited — such as allocations that only need to live until the end of the current frame — the whole allocator can be reset to reuse the memory, as the Rust compiler's borrow checker can ensure that no references to the allocator remain. In specific terms, this was implemented by the allocation function being accessible via an immutable borrow of the allocator, which enables sharing the allocator

around freely for allocating memory from it, and the reset function being accessible via a mutable borrow, which only allows resetting after all of the immutable borrows have been dropped.

Since the linear allocator could only free memory with a mutable borrow, the design was appropriate for two kinds of allocations: ones whose lifetimes are simple enough to be borrow checked, and ones whose lifetimes span the whole lifetime of the program, and thus do not need to be freed. This makes the linear allocator ideal for both persistent allocations and frame-internal allocations, but leaves a need for another kind of allocator, which could be used for allocations that do need to be freed at some point during the lifetime of the program, but whose lifetimes are too dynamic to handle with the borrow checker. This was needed, at the very least, for loading in different assets for different parts of games. Interestingly, even though this design was largely motivated by the limits of what kinds of lifetimes can be handled by the Rust borrow checker, the exact same taxonomy — persistent memory, one-frame memory, and dynamic memory — has been described in a 2008 blog post by Gyrling about having separate memory pools for each of these three kinds of lifetimes [10].

The resource subsystem was designed along the lines of the chunked resource database described in *Game Engine Architecture, 3rd Edition* [9, pp. 513–514]. The core idea was that any assets needed by the game at any particular point in time could be loaded in small chunks, and only the relevant chunks would ever need to be loaded at the same time, smoothing out the resource loading process to match the actual resource usage of the game. As mentioned in the book, this requires the assets to be processed into a chunked format beforehand, which was handled by creating a command line tool for importing images and audio files into a *resource database file* that could be read by the engine, and which would contain those files' data in a chunked format that the engine could load as needed. The resource database file format has metadata at the start, which is loaded into memory at startup in its entirety, and the actual chunk data after, which is loaded as needed. Individual assets are represented in code by handles, queried from the resource database based on unique names. The handles can be queried when the game starts, and used later to make use of those assets, and the engine handles loading in the appropriate chunks based on the asset metadata. These handles are indices to arrays of asset metadata, but stored inside a newtype to avoid out of bounds accesses, which would result in a panic.

For the resource subsystem of the engine, some kind of dynamic allocator with support for reusing memory without clearly delineated lifetimes was required, and a pool allocator

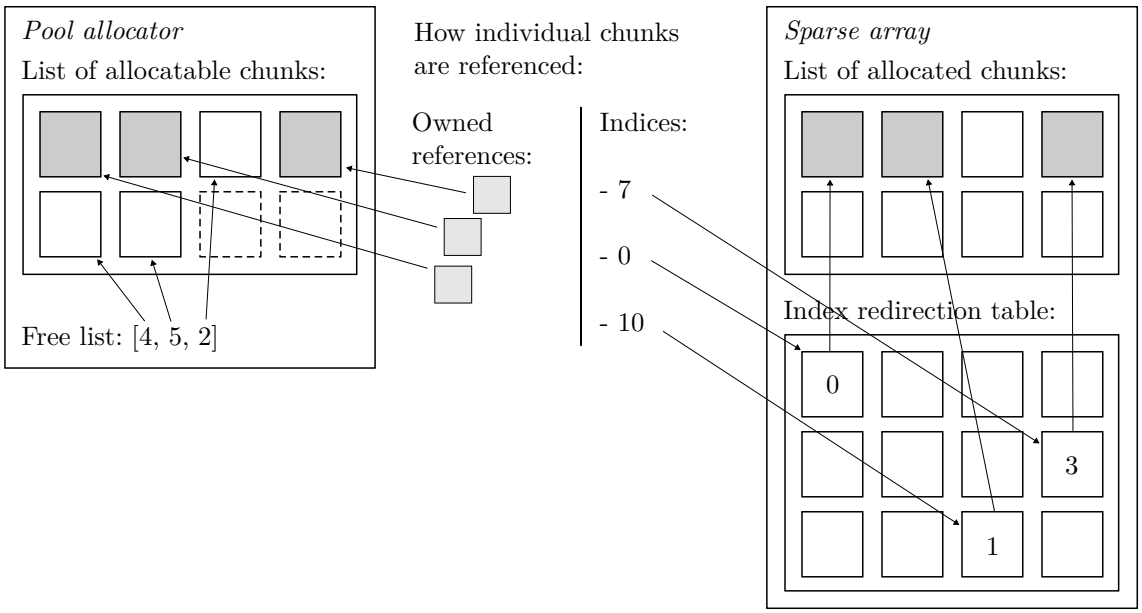

Figure 4.4: This diagram highlights the differences between how chunks of data are referenced between the originally planned pool allocator, and the sparse array that ended up being used. The un-filled chunks represent memory that has been allocated, but is not in use, while the dashed rectangles represent potential for allocating more memory from the OS if needed.

was deemed the most simple. A pool allocator would be split into constant size chunks, and chunks could be allocated from a free list of unused chunks, and dropping a chunk would return it to the free list for reallocation. This design ran into development issues, and was replaced with a sparse array of chunks instead. The sparse array consists of an index redirection list, and a list of loaded chunks — that is, constant size allocations representing the runtime data for a given chunk that resources could refer to. The index redirection list has some performance and memory overhead, and references to the sparse array cannot live for very long — as new chunks cannot be allocated while others are being read — but this design was simple to implement and worked for the purposes of the resource subsystem.

The differences between the pool allocator and sparse array are illustrated in Figure 4.4. The main technical obstacle the sparse array solves is that it uses indices, and owns its memory — compared to the pool allocator, where the allocations themselves would have to own their memory. Indices pointing directly to the allocated chunks could have been used in a more direct replacement for the pool allocator. However, since every chunk's metadata was stored in a *chunk descriptor*, which themselves were stored in an array in the resource database, the concept of a sparse array was considered a good fit: the sparse array would represent all of the resource database's chunks, even unloaded ones, and the

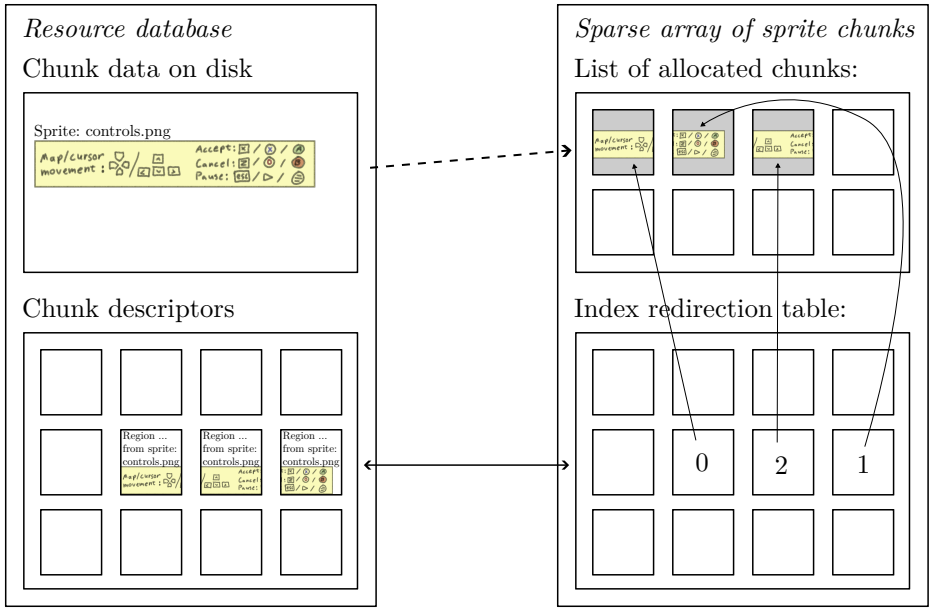

Figure 4.5: Each asset in the database consists of a range of *chunk descriptors* — which contain, for example, where the image is in the data section of the on-disk resource database — which can be loaded one at a time, and crucially, are loaded into the same index in the sparse array, as the chunk descriptor's own index in the resource database's array of chunk descriptors.

index redirection table would work as the mapping from chunk descriptor indices to the allocations where their data was stored, as illustrated in Figure 4.5. A key detail here is that most of the chunks in the resource database are not loaded most of the time, so the backing memory of the sparse array can be much smaller than what would be required to hold everything in memory at once. Storing an index for every chunk was not too expensive either, since every chunk represented 64 KiB of memory when loaded, so there would need to be terabytes of game assets before just the index storage would get out of hand. If the engine ended up using a pool allocator instead, there would probably have been some kind of layer for mapping chunk descriptor indices to the chunks allocated from the pool allocator as well, so in the end, a whole another generic allocator just for allocations with more dynamic lifetimes seems to have been less necessary than initially expected, at least for this particular use case.

The rendering and audio playback subsystems were designed to take in asset handles, pick the appropriate chunks, and render or play back those chunks. The rendering API consists of a drawing function that can be called using *sprite assets*, which gets converted into chunks by calculating the size of the sprite on the screen compared to its resolution, picking the appropriate version of the sprite for that resolution, and recording two triangles with appropriate texture coordinates in a draw list. Similarly, the audio API is exposed

via the `Mixer` type, which can take *audio clip assets* for playback, which it stores in a list of currently playing sounds, and each frame, loads and uses the samples from the appropriate chunk for playing back the sound. Both the rendering and audio system dispatch triangles and audio samples respectively to the platform layer at the end of each frame, which the platform implementation library is then responsible for actually drawing on screen or playing back from the speakers.

Since the prototype design intentionally omitted any specific gameplay systems, but some kind of framing for gameplay logic would be appropriate to avoid games having to allocate their own memory for gameplay constructs, an entity-component-system (ECS) was included. The ECS is organized around *scenes*, *game objects*, and *components*. The scenes are containers for game objects, and each game object is conceptually a container of components, and the components actually contain the data relevant to the gameplay code. As ECS architectures were previously described in Section 2.3, this system implements a property-centric runtime game object model. Game objects are represented by structures that implement the `GameObject` trait, which provides some functions that a `Scene` can use to allocate properly aligned and sized buffers for each component type across all game object types that are associated with the scene. Memory is allocated per component type to help with data locality, and there is an API for operating on all game objects that have a particular combination of components, which is used to fulfill the *system* role in ECS terms.

4.2 Development process

The linear allocator was the first major component implemented, as nearly all other game engine subsystems need the ability to allocate dynamic amounts of memory. As described in Section 2.3, a linear allocator is a kind of allocator which allocates memory by incrementing an offset into its backing memory, considering all memory before the offset to be in use. Since there's no metadata or free list for tracking freed allocations, the only way to reuse memory is to reset the whole allocator after no allocations are in use anymore. In Rust, this was easily modeled with the borrow checker: allocations can be made with an immutable borrow of the allocator, while resetting requires a mutable borrow. Since the borrow checker makes sure that immutable borrows can't exist at the same time as a mutable borrow, and that there is only ever a single mutable borrow, the reset operation of the allocator is guaranteed to be safe, and not cause use-after-free bugs. As seen in

```rust
let mut frame_alloc = ...;
loop {
  // No references to ‘frame_alloc‘ at this point, so we can access it mutably.
  // For example, to reset it to free the previous frame's allocations:
  frame_alloc.reset();

  // During the frame, the allocator can be used to allocate data that only needs to live one frame.
  let mut frame_data = frame_alloc.allocate(...);
  simulate_game(&mut frame_data);
  let mut draw_calls = frame_alloc.allocate(...);
  render_game(&mut draw_calls);

  // ‘frame_alloc‘ borrows are dropped here, as ‘frame_data‘ and ‘draw_calls‘ drop out of scope.
}
```

Figure 4.6: As seen here, linear allocators and the borrow checker work especially well for data that is needed in a clear scope — such as one iteration of the game loop. The borrow checker ensures at compile time that the allocations made from the frame arena do not leak, by returning an error about the allocated object outliving the reference to the allocator, or by pointing to the allocator's reset function as not being valid, due to shared references of the frame allocator still existing.

Figure 4.6, this pattern works well on the caller side when the lifetime of the allocations are clear, such as one iteration of the game loop. On the other hand, since all the allocations are represented by a borrow of data from the allocator, all types containing these allocations need to have a generic lifetime argument, which does lead to somewhat noisier looking code. This lifetime argument is bound at the location of the allocation operation to the lifetime of the immutable borrow of the allocator, which can effectively be one of two: the lifetime of a specific scope, such as a function's body or one iteration of a loop, or static, the length of the rest of the program. Static allocations effectively lock the allocator to never be reset again, but this use case is surprisingly relevant to game engine internals, as briefly mentioned in Section 4.1: many subsystems need dynamically allocated memory, but the required amount is known when the engine starts up, so the allocations can be made once and used for the rest of the program — these allocations make up the *persistent* memory discussed in the blog post by Gyrling [10].

The internals of the linear allocator were implemented using some `unsafe` code. In particular, to allow allocating objects of different types, the backing memory is represented by a pointer which is incremented and aligned to acquire the pointer for the allocation. To make this pointer usable in safe Rust code, it is turned into a regular mutable bor-

row, which is an unsafe operation, since it requires upholding some invariants to avoid undefined behavior. Validating this unsafe code took time, but was made plausible thanks to two resources: the Rust standard library documentation, which lists these invariants clearly, and Miri [19], an interpreter for Rust which checks the code it is running for violations of these invariants at runtime. Likely due to the simplicity of a linear allocator, no safety violations were found with Miri, but the much more complicated multithreading implementation had multiple issues it detected.

There was an attempt to make a pool allocator for dynamic allocations with lifetimes that could not easily be modeled for the borrow checker. The core idea was to make an allocator whose backing memory had a generic lifetime, which could be static, but which also had the ability to reclaim memory when allocated chunks were dropped. This effort ran into difficulties when trying to share access to the pool's free list among all allocations, so that the allocations' drop implementation could return the chunk "back into the pool" by appending it into the free list. Both a linked list and an array-based list were attempted, and internal mutability was used to allow sharing multiple references to it, but ultimately the problem came down to not being able to parameterize the lifetimes so that the borrow of the free list among the allocations did not bind the allocations to each other. Instead of trying to add more runtime machinery or the unsafe escape hatch for this, a `SparseArray` container type was implemented for the purposes of the resource management subsystem.

In general, the lifetimes of allocators in the engine caused much complication in the beginning, due to structs not being able to contain borrows of data contained within the same struct. This resulted in the core `Engine` type offloading a lot of initialization outside itself, so that it could borrow those values, as they were owned outside of it. Eventually most of this was cleared up by compromising on how dynamic the engine's lifetimes were, and simply using static lifetimes for borrows that live the whole lifetime of the engine. The cost for using static lifetimes is that it makes reinitializing the `Engine` type within the same process problematic, as the previous Engine's borrows need to stay valid even if it is dropped, due to the borrows having a static lifetime.

The resource management subsystem in the prototype was split into two parts: the `ResourceDatabase`, which holds all currently loaded data and is immutably borrowed across game-specific code to make use of the game assets, and the `ResourceLoader`, which holds a queue of loading requests for assets that need to be loaded. The loader starts asynchronous file loading operations, and processes any finished reads once a frame to update the database with loaded data for the assets. The memory layout of the actual loaded

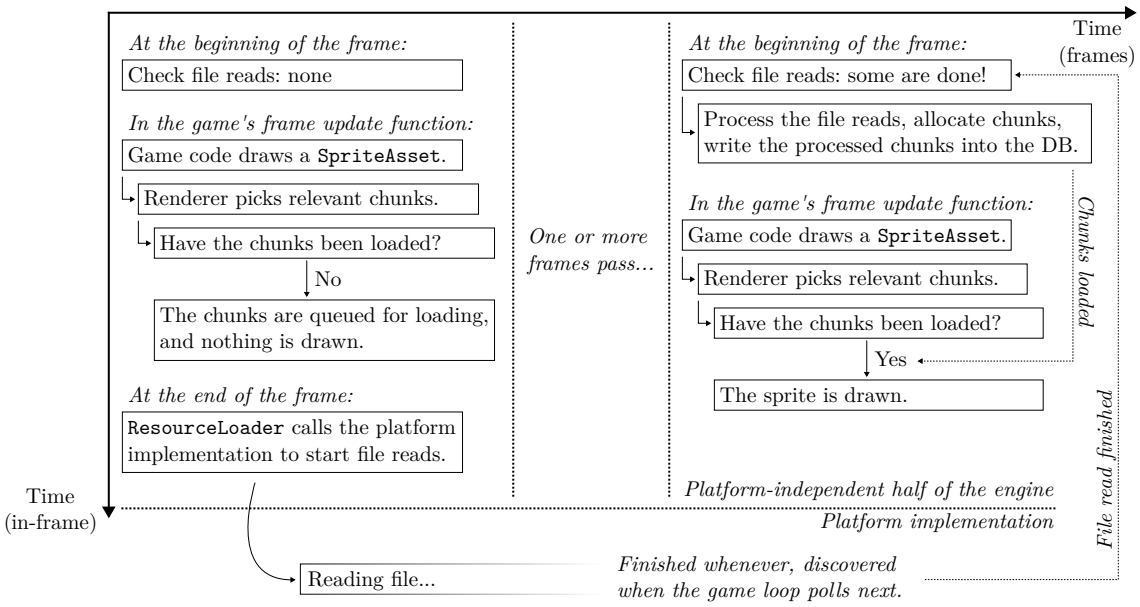

Figure 4.7: The general flow of the game asset streaming system, using sprite rendering as an example. Note that operations in the game's frame update function do not need mutable access to the `ResourceDatabase` — the API is specifically designed to allow game code to hold shared borrows of it for the entire duration of the frame, only `ResourceLoader` needs to be borrowed mutably for queuing chunk loads. However, in the beginning and end of the frame, `ResourceDatabase` *is* mutably accessible, and this access is used in particular to update chunk data as they are loaded.

assets was designed around *Pool-Based Resource Allocation* described in *Game Engine Architecture, 3rd Edition* [9, pp. 513–514]: as part of the asset conditioning pipeline, game assets are split into chunks to make them fit in constant-size chunks of memory, and on the runtime side, assets are loaded by-chunk, so that unneeded chunks are not unnecessarily loaded. The pool of chunks in `ResourceDatabase` uses the aforementioned `SparseArray` container to hold the chunk data by assigning each chunk a unique index offline, and those indices are used to refer to the chunks in asset metadata. Using indices leads to the need to check if the chunk at a given index is actually loaded when accessing it, but avoids lifetime management issues due to not being an actual reference.

The split between `ResourceDatabase` and `ResourceLoader` arose from the observation that the resource management subsystem really consists of two halves with different memory access patterns. On one hand, asset data needs to be read, for example, to draw a sprite, and those reads are represented by borrows which may need to live for a while, as the sprite might need to be drawn multiple times, possibly even interleaved with other sprites being drawn. On the other hand, if the chunk containing the sprite's pixels is not yet loaded, a loading request should be pushed to a loading queue. Pushing onto a

queue implies mutation, but the mutable borrow only needs to last for the duration of the push operation. Based on these two access patterns, `ResourceDatabase` was designed to hold the loaded data, which can be immutably borrowed many times for use throughout the current frame, and `ResourceLoader` was designed to hold the loading queue, which is not borrowed for later use, but rather only borrowed momentarily per-draw, to receive any possible loading requests. It should be noted that the loaded data does need to be updated occasionally with data from finished loading requests, but this can be done outside of game-specific frame processing, which means that there are no immutable borrows of `ResourceDatabase` that would interfere with the update. This general design around setting up queues of asynchronous operations, which can be pushed onto during regular frame processing, and popped to process that data with full mutable access to the engine, ended up simplifying a lot of questions around mutable and immutable borrows between different parts of the engine. An example of this pattern can be seen illustrated in Figure 4.7, which shows how this scheme works in practice when rendering sprites and asynchronously loading them from disk.

On the other hand, sometimes mutable access is needed even though the structure does not do any mutation from the perspective of the caller. This is the case in the implementation details of `SparseArray`, which maintains a list of numbers representing the amount of frames since each element was last read, which is used to evict the least recently used element when the array's backing memory becomes full and new elements need space. Since reading an element should be an immutable operation, and the read is also the operation which should update the access time, there is no clear split that could be made like in the case of `ResourceDatabase` and `ResourceLoader`. For these cases, interior mutability is needed, which is provided in the core library via the `Cell` and `RefCell` types, as well as the atomic variants of some primitive types, which allow temporary mutable access via an immutable borrow with some caveats: the cells cannot be shared between threads, and the atomic types can only contain a single value of a small set of primitive types, and atomic operations have a performance cost.

The multithreading utilities were similar in development to the allocator: unsafe core components with a lot of manual review, but simple semantics to keep that code as simple as possible. Since the Rust standard library's core does not include any multithreading utilities due to them often being platform-dependent, a lot of the multithreading code resides in the shared platform library, in addition to the platform implementation. In particular, the shared platform library — otherwise mostly containing only data types

to allow the platform implementation and engine to communicate — contains a single-producer single-consumer channel for communication between threads, and a thread pool type for spawning and joining tasks on worker threads. Similar to the unsafe operations in the allocator, the standard library also contains good information on the `Send` and `Sync` traits which define what is safe to send across thread boundaries, and the atomic operations used for synchronization.

Even with the simple design, the multithreading utilities ended up being more complex than the memory allocator, and had a few bugs that were not caught right away. However, when the multithreading utilities were integrated into other parts of the engine, which were tested with Miri, some bugs were caught. In particular, in the initial design, the main interface to the thread pool was designed to be similar to the Rust standard library's scoped threads. Scoped threads are used by passing a closure into the `scope` function from the `std::thread` module, and in the closure, new threads can be spawned to do work, and joined to receive the results. In particular, these new threads can borrow values from the surrounding context for the length of the `scope` call, unlike the general purpose thread spawning function, which can only borrow values for the static lifetime. The main difference to the standard library's design was that the thread spawning function would take a mutable slice of data to process, spawning many tasks on the thread pool to process that data in many parts — "scattering" — and joining all those tasks when the results are needed — "gathering." This ran into a fundamental restriction of the borrow checker: once a mutable borrow is split for scattering, the split parts cannot be recombined back into one mutable borrow for gathering. Neither of the theoretical models for borrows implemented in Miri — Stacked Borrows [11] and Tree Borrows [24] — allow for this, based on the errors reported by Miri. Since the scatter operation could not be gathered, the final design simply combined the two into a single `parallelize` call, which avoids the need to recombine the mutable slices. This works by splitting the borrow to do the work, but instead of combining the split parts, the `parallelize` function simply returns after the work is done, releasing the mutable borrow that was passed in. This does require the main thread to be blocked doing nothing while waiting for the worker threads to finish, and does not allow queuing up other work ahead of time, which can waste utilization on cores that finish early.

Another issue faced while developing the multithreading utilities was the so called *leakapocalypse*. In most cases, Rust values call their `Drop` implementation when they fall out of scope, where resources can be cleaned up. However, when values fall out of scope, they are

```rust
// First, initialize some memory where the file is read into.
let mut file_read_dst = [0; 1024];
// Start reading the file. This passes the pointer to the platform implementation, which has been
// ‘‘proven safe’’ based on ‘file_read_dst‘ being mutably borrowed by ‘file_read_task‘ until
// the task is finished or dropped.
let file_read_task = start_reading_file(&mut file_read_dst);
// Drop ‘file_read_task‘ without ‘Drop::drop‘ being called for it. This is safe, because the
// leakapocalypse proved that this can be done with safe Rust anyway.
core::mem::forget(file_read_task);
// The file read has not finished yet, so the platform still has mutable access to ‘file_read_dst‘,
// but since the task is no longer in scope, ‘file_read_dst‘ is no longer mutably borrowed by the task,
// and thus is mutable again here.
file_read_dst[0] = 123;
// Mutating ‘file_read_dst‘ while the platform still has mutable access to it is undefined behavior:
// the ‘start_reading_file‘ proof must be unsound.
```

Figure 4.8: Minimal example of how the unsoundness of the first version of the file reading interface could be exploited to cause undefined behavior. This was solved by using an *owned* `Box` for the destination memory instead of a *borrowed* byte slice — with owned destination memory, the memory needs to be explicitly reclaimed via a function that can ensure that the platform's file read is done, instead of being implicitly reclaimed by the file reading task going out of scope.

not *guaranteed* to run the `Drop` implementation. This caused a major unsoundness bug in pre-1.0 Rust related to scoped threads [3], which manifested in two places while implementing this game engine prototype: the initial scatter/gather multithreading interface, and the original file reading interface followed a similar pattern to pre-1.0 Rust scoped threads. The problem manifested as follows: there is some asynchronous work to do, which needs mutable access to some memory. To provide a neat interface for this, a task type is created, which holds the mutable borrow as a private field, and does not provide any methods to mutate it. This task type is then considered to own the mutable borrow from the interface perspective. Internally, since this task type does not make any use of the mutable borrow, it turns the borrow into a pointer, and passes it to some external system that does the asynchronous work. Since the task struct contains a lifetime parameter to which the mutable borrow is bound, it can only be used by the external system for as long as the task object exists. In the expected flow, the interface user would later call a join function to wait for the asynchronous task to finish, and receive back the mutable borrow, now with data processed by the external system. In the case that the join function is never called, the task type would drop out of scope, and the mutable borrow would be released, which would cause undefined behavior: since the external system is still working on the

data and accessing it via the pointer it got, this mutable borrow would not be exclusive, which is unsound. To "fix" this, the `Drop` implementation of the task type could wait for the external system to finish its work, ensuring that the mutable pointer shared to it is no longer in use by the time the value goes out of scope and the mutable borrow released. This is the key issue: as seen in Figure 4.8, due to Rust allowing values going out of scope without their drop implementation being ran, this fix does not solve the issue. In practice, this means that asynchronous tasks simply need to be modeled differently, for example by using owned memory like a `Box` or `Vec`, instead of borrowing it.

Without `Box` or `Vec`, "owned memory" can be represented with a mutable pointer and some carefully designed invariants for using it. In the prototype, these were used to implement the linear allocator, which attaches a lifetime to the allocations made from it to make sure the allocator stays immutably borrowed as long as the allocation exists, and a ring buffer, which is used for short-lived but dynamic allocations. The ring buffer is used to provide the backing memory for asynchronous file reads and thread pool tasks. It was designed to be similar to the linear allocator implemented earlier, except that allocations could be freed, as long as they were freed in the same order they were allocated. This involved adding some tracking metadata alongside the allocations, which could be used to ensure that a particular allocation really is the oldest at any particular point in time, which when passed back to the ring buffer, would advance its internal allocation offset to make space for more allocations.

While some core systems of the engine used a raw pointer for representing owned allocations, in safe contexts, owned memory in the engine was represented with a custom `Box` type. The standard library version from `alloc` was not used as it revolves around the *resource acquisition is initialization* (RAII) concept, meaning that upon drop, the memory is freed, which was not desirable for the kinds of allocations that needed representing in this engine — the memory never needs to be freed anyway, or should be reused, ideally for the same purpose. The custom `Box` provides two constructors for these use cases: a safe one, turning a static mutable borrow into a `Box`, and an unsafe one, turning a raw pointer into a `Box`, requiring the caller to ensure that the pointer is never written, read, nor freed after the creation of the `Box`. However, the memory can still be reused by consuming the `Box` to turn it back into a pointer with a function called `Box::into_ptr`. This kind of `Box` abstraction allows safely handling dynamically allocated, owned objects, without needing RAII style allocation and freeing for each one. The downside to this approach is that once a `Box` is created, the memory referenced by it cannot be reused safely without

reclaiming it: this results, for example, in the ring buffer having the potential to end up locked, if even a single allocation from it is lost — since the memory owned by the lost `Box` cannot be reclaimed via `Box::into_ptr`, the ring buffer can not be sure that reusing that memory is safe, and thus, the ring buffer is stuck, unable to allocate after the next allocation would need to reuse that part of memory. A more sophisticated allocator would be able to still reuse the rest of the memory, but a ring buffer was simpler to implement and avoids any related memory and performance overhead.

As the final major part of the engine, an asset conditioning pipeline was implemented in the form of a command-line tool called `import-asset`, which was written for offline development use. As such, its compilation time and platform-specific dependencies were less of a concern, and it makes wide use of the whole Rust standard library, as well as many libraries for parsing various kinds of files. Notably, it was much less work compared to the rest of the engine, perhaps in part due to it being simpler software in general, but likely also due to not needing to pay attention to the lifetimes of allocations and other low-level details, thanks to the standard library types that abstract over those details.

In order to maximize Miri testing coverage of the unsafe parts of the engine, alongside unit tests, the prototype included a sort of integration test for the platform-independent part of the entire engine, called the "smoke test." This test initialized the engine as a game would, and ran the game loop for 300 iterations — simulating 10 seconds of a "game" running at 30 frames per second — sending button presses to the engine, rendering some sprites, and playing some sounds based on those buttons. The duration was later reduced to 40 iterations — 4 seconds at 10 frames per second — as the smoke test ended up taking over 40 minutes and saturating the GitHub Actions CI quota, after audio mixing was added. That said, the game engine still processes frames in well under a millisecond when compiled in release mode, Miri is just inherently slow due to interpreting the code, and not being able to optimize it at all to avoid optimizing out soundness issues. Normally interactive applications like this are somewhat hard to run through Miri not only due to performance, but Miri also sandboxes the entire process and does not expose most operating system level APIs or allow linking to dynamic libraries. In this case, it was possible due to the clear delineation between platform-independent and platform-specific code, and the smoke test could use a separate platform implementation meant specifically to run in Miri, and for testing in general. This testing platform implementation had its own time counter instead of using actual wall-clock time, could simulate inputs, and implemented rendering and audio playback as no-ops.

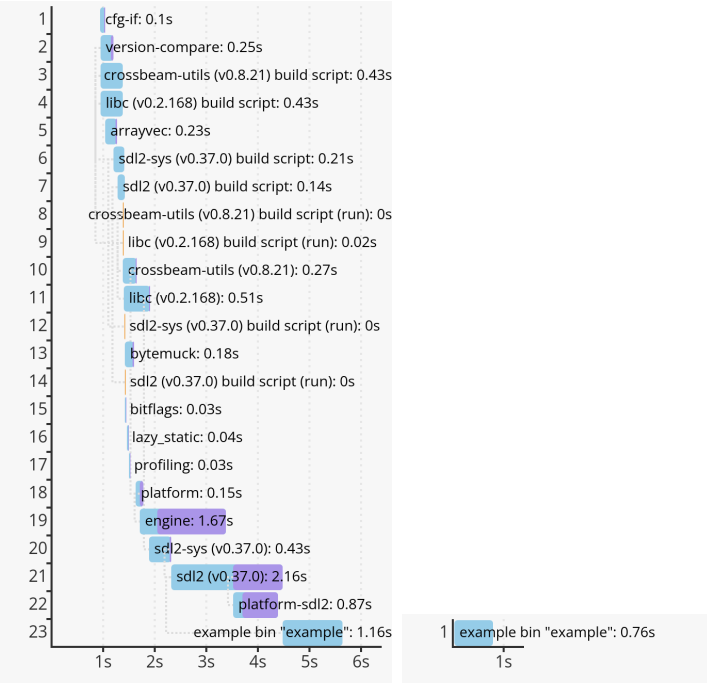

Figure 4.9: The compilation timings for the engine's example binary, with a non-incremental release build on the left and an incremental debug build on the right, generated by running `cargo build --package=example --timings` (with and without `--release`) in GitHub Actions CI for the prototype game engine's commit `64905041f12b24e0da5fdbc6733bd3152b685b10`.

Regarding the measures taken to keep compile times low, the general picture is good: as seen in Figure 4.9, a non-incremental build of a small example game with release optimizations enabled takes a bit under 6 seconds, and less than a second for an incremental one without optimizations. While it is hard to say if avoiding monomorphization helped due to lacking a point of comparison, at least avoiding procedural macros seems to have helped, as the `engine` crate starts building within the first second, after crates start being built at all — the first crate of the graph starts building at the 1 second mark, likely due to some general setup at the start — and building `proc-macro2` and `syn` would likely have taken more than a second in GitHub Actions, increasing build times. However, what was anticipated to cause the greatest gains, did not materialize due to the engine compiling too fast: parallelizing the compilation of the platform implementation and the rest of the engine. Since the `platform-sdl2` crate depends on `sdl2`, which takes longer to build than `engine`, it ends up being built late enough that the `engine` crate has already finished, which means that the build time did not end up benefiting from their split. The benefits would start appearing if the `engine` crate expanded enough to take more than about two seconds to build, or if the platform implementation's dependencies were faster to build.

5 Results

This chapter presents observations and time tracking data from the development process, and evaluates the artifact developed as described in the previous chapter. Section 5.1 highlights practical observations from the development process, Section 5.2 presents the results of the time tracking done during development, Section 5.3 describes the game built on top of the artifact for demonstration purposes, and Section 5.4 evaluates the artifact by comparing it to the objectives set in Section 3.2.

5.1 Observations from the development process

This section highlights some observations from the development process, which was described in more detail in Section 4.2.

Observation 1: *Rust's safety guarantees in multithreaded code can apply even when not using the standard library multithreading implementation.*

Early on in designing the multithreading utilities of the engine, it became clear that the `Sync` and `Send` traits, combined with the borrow checking semantics between shared and exclusive borrows, are enough to be able to implement a good, safe multithreading API — the standard library threading utilities really are more like a well-tested and proven library, rather than something fundamental to the language itself. This means that it can be replaced with another one, such as the platform abstraction used for this engine, and such a library can still provide an API that makes sure that unsynchronized mutation does not happen.

Observation 2: *not all allocations in a game engine can be represented as a borrow with a specific lifetime, but a lot of them can.*

Aside from the resource database's streamed resource data chunks and the ring buffers used by asynchronous operations, all of the allocations in the artifact are done using a linear allocator. Some use the "persistent arena," which never gets reset, and thus can provide allocations that live for the static lifetime, and others use the "frame arena," which gets reset at the beginning of each frame. These two lifetimes are clearly defined, so modeling the allocated memory as a borrow with a specific lifetime fits these use cases.

Observation 3: *types representing data with a highly dynamic lifetime should avoid generic lifetime parameters, and instead rely on either "boxes" or pointers to refer to the data.*

As mentioned above, the resource database streaming, the thread pool's tasks, and the file reading buffers do not make use of the linear allocator, and that is because they cannot be modeled using borrows. When it comes to representing asynchronous tasks, which is what these systems require, there is always the possibility that the task is never finished, whether due to the task actually never finishing, or the other side never "taking" the results. To allow for that, any allocation represented by a borrow would necessarily need to be borrowed for the static lifetime. This could be fine if the asynchronous operations only happened, for example, during the engine initialization phase, as it would be a predictable amount of memory that is effectively leaked after the operation finishes — since the borrow would be static, the allocator could never reuse that memory. However, these systems are constantly running, and these asynchronous operations happen every frame, so the memory must be reusable. To summarize: some use cases need to allocate memory that can be reused, but could also live forever — there is no lifetime that can accommodate both of these, so these cases need to own their memory, by using something like the standard library's `Box` type.

Observation 4: *the design of data structures in Rust depend heavily on which parts of it need to be mutated and when.*

As mentioned in Section 4.2, the resource database ended up being split into two types to allow sharing the main database structure around, while concurrently mutating the loader — a separate structure, split off from the main database structure — for queuing up mutations to be later applied back to the database structure. This pattern proved useful, as requiring a mutable borrow heavily affects usage patterns: mainly, it leads to using the data in "one-shot" operations, to avoid having to store the mutable reference for later. Storing borrows for later is often useful, and they can be mutable, but when mutable borrows are interleaved with other code needing access to the same data, the borrow checker will quickly get in the way, so immutable borrows are preferable. This pattern applies to game engines perhaps especially well, since are structured around the game loop, which means that any temporary borrows do not last very long.

Observation 5: *implementing asynchronous operations is hard due to some slightly obscure Rust semantics. Consider using an established solution from the library ecosystem instead.*

The prototype had an unsound file reading abstraction for a large part of development, as

Refactoring	Lifetimes	Ownership	Unsafe	All tasks
54h 2m	112h 15m	57h 19m	49h 21m	236h 8m
22.9%	47.5%	24.3%	20.9%	100.0%

Table 5.1: The first four columns show how much time was spent on each topic of concern during the development process, and how large a percentage of the full development process that time represents. The last column accounts for all tasks, for reference.

it made wrong assumptions about `Drop`, a case of the leakapocalypse — described in more detail in Section 4.2 — and it was not immediately obvious. It was only detected after the same pattern was used in the multithreading implementation, which was only detected when tested with Miri, specifically when testing with multiple simulated threads. Due to this being such an easy issue to overlook and hard to detect, it would have been better in retrospect to not implement the asynchronous systems — file reading and multithreading primitives — as part of the game engine.

5.2 Time tracking

The development process of the prototype game engine was organized by dividing work into tasks, and time spent on each task was tracked by noting the starting and ending times of continuous work on each task, and summing up these chunks of time spent. The time includes time spent on designing the particular implementations, but not the entire design process. Each task was also associated with one or more parts of the engine that the task was concerning, as well as the nature of the task, and whether some Rust-specific concerns were relevant to that task.

The parts of the engine tracked by the tasks were picked from the high-level parts described in Figure 2.1, like the *Low-Level Renderer* or *Gameplay Foundations*. To fit the data into less columns — making analysis easier and display of the full table plausible — tasks that touched on multiple different parts of the engine were split into multiple rows of time tracking data, one row for each engine part, and the time spent was split equally between those rows.

The raw time tracking data with individual tasks and all of the data points related to each task can be found in Appendix A.

The Rust-specific concerns that were tracked when working on the tasks were categorized

	Refactoring	**Lifetimes**	**Ownership**	**Unsafe**
3rd Party SDKs	0.0 %	0.0 %	2.3 %	0.0 %
Asset Conditioning Pipeline	0.0 %	0.0 %	0.0 %	0.0 %
Audio	0.0 %	6.3 %	0.0 %	0.0 %
Core Systems	43.3 %	19.8 %	39.9 %	59.6 %
Gameplay Foundations	3.0 %	0.0 %	0.0 %	0.0 %
HID	0.0 %	0.0 %	0.0 %	0.0 %
Low-Level Renderer	5.6 %	0.0 %	0.0 %	0.0 %
Platform Independence Layer	12.6 %	19.1 %	10.5 %	40.4 %
Profiling & Debugging	0.0 %	0.0 %	0.0 %	0.0 %
Resources (Game Assets)	35.4 %	54.8 %	47.3 %	0.0 %

Table 5.2: The first column lists out the parts of the engine that were included in the engine — based on the parts seen in Figure 2.1 — and the rest describe how the development time was split between those parts, one column for each topic of interest. For example, out of all the tasks dealing with `unsafe` code, over half were tasks developing the Core Systems section of the engine, while the remaining 40.4 % were about working on the Platform Independence Layer.

as lifetimes, ownership, refactoring, and dealing with `unsafe` code. As mentioned in Section 2.2, the concepts of lifetimes and ownership, which need to be considered when designing algorithms and data structures, are a common point of friction when learning Rust. The amount of refactoring needed was also cited as a reason making Rust less suitable for game development specifically, due to adding friction to iteration-focused development. While refactoring would be more of a concern when writing game-specific code, rather than engines, it was tracked as a potential source of friction. Finally, code marked with the `unsafe` keyword inherently carries with it a source of friction: it requires the developer to manually ensure that the code is safe, allowing the expression of more complex patterns, at the price of not getting as much compiler validation. The amount of time spent on these topics can be seen in Table 5.1, and in a similar finding to the paper by Zhu et al. [27], lifetimes were the most relevant topic across all tasks — which could be due to difficulty as in the aforementioned paper, but lifetimes may also simply be the most relevant concept to think about when working on this particular use case in Rust, out of the four topics tracked.

Table 5.2 shows how time spent on each of the Rust-specific topics of concern is split between the parts of the prototype game engine. The *refactoring* column is perhaps the least surprising, as the *Core Systems* are by nature depended on by almost everything else, and thus would require the most refactoring, as requirements change during the

development process. Time spent designing *lifetimes* and solving issues related to them is heavily focused on the *Resources* subsystem, which is likely due to it including the most dynamic data structures in the engine, and a lot of time was spent trying to reconcile them with Rust's lifetime annotations, before giving up, as reflected by Observation 2. The split of *ownership* related tasks is still focused on *Core Systems* and *Resources*, likely due to these systems containing most of the engine's data, and thus their design would require thought on where that data should live. Perhaps most interestingly, the *unsafe* column highlights how, even when needed, code marked with the `unsafe` keyword can be focused in low-level primitives, that the rest of the codebase can then use safely — more than half of the time spent on tasks related to unsafe code was spent working on the *Core Systems*, which include both multithreading and allocators, the two main systems requiring extensive `unsafe` usage. The remaining time spent on unsafe code in tasks related to the *Platform Independence Layer* makes sense, as dealing with platform-specific functionality often requires unsafe code, such as calling non-Rust libraries and using data received from them, and due to the `platform` crate including some synchronization primitives.

5.3 Demonstration

To validate that the prototype game engine could actually serve the purpose of a game engine to some degree, it was used to develop a game during the Ludum Dare 57 game jam — specifically the "Jam" variant of the event — called "Diving for Oxygen." This jam spanned 72 hours, during which the participants would make small game based on a shared theme, using the tools of their choice. While a game jam does not reflect many of the difficulties of developing a commercial game over a longer development period — importantly having a lower bar of quality and complexity — they do simulate some important parts of the process from beginning to end. This section will describe the game, its implementation, issues and lacking features found within the engine, and finally, whether the prototype game engine fit its purpose, by providing a usable base for a small game.

The game was built with a similar technical base as the engine itself: split into two crates, one for producing the binary and managing the game loop, and the other for the actual game logic and engine interactions. The library side does not use most of the standard library much like the engine, using the `#![no_std]` annotation. Aside from the engine and some dependencies shared with the engine — such as `bytemuck` and `arrayvec` —

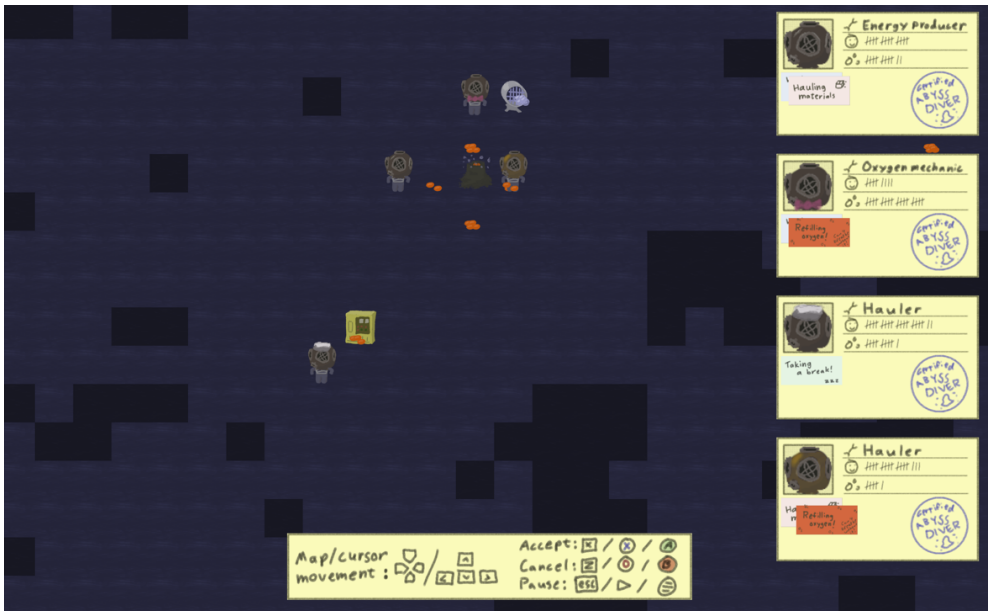

Figure 5.1: Screenshot of the demonstration game, using the prototype game engine. The characters move around autonomously in the world, pathfinding their way around each other and the darker parts of the grid-based world. The "status pages" on the right show what each character is doing and some statistics that can explain why they are doing that.

the game had a few additional dependencies: the `glam` crate for linear algebra types and operations, and `seahash` for seeding the noise functions used in the game.

Regarding game design, the game is a colony simulation game, where the player takes the role of a manager, assigning work for a few simulated characters, which act on those orders based on the state of the game world. Figure 5.1 shows how the world is shown to the player, as well as the status cards which show some details about each character. The game world can be roughly split into three parts: the environment, resources, and characters. The environment consists of a 2D grid which the characters can find paths in and move through, the resources represent the more dynamic, character-usable parts of the environment, and the characters act in the environment based on their programmed behaviors and simulated state. A large part of the game's codebase deals with simulating the behaviors of the characters, depending on their character-specific state, as well as the state of the environment around them. There are a few different resources, which the characters can move around, transform into other resources, and consume. This sort of simulation game could be considered an ideal fit for the architecture implemented by the prototype engine's scene system, as there is no one part of the game where only a single entity of a given type would exist — there are many characters and many resources, so the architecture focused on handling entire collections of components at a time fits well.

Even though the scene system otherwise fit the game well, it requires the game objects' components to implement the `Zeroable` and `Pod` traits from the `bytemuck` crate — to allow manipulating the game object data in various ways without unsafe code — which turned out to be a problematic restriction when it came to the data structures used to represent the internal behavioral state of the characters. The problem is that the `Pod` trait cannot be implemented for types which contain enums, which were very useful for implementing the behavior system due to being able to contain different types of data in one region of memory, depending on the variant of the enum. To be able to use enums to represent this behavioral state, the game object side ended up having a component that contains an index, which could then be used to index into a regular array on the game's side, where the actual behavior state was stored. This side-stepped the problem, but also brought into question the usefulness of the scene system of the engine in the first place: why not simply store the components on the game side, in a way appropriate for the particular game?

Another shortcoming of the scene system of the engine came up when implementing collisions: the naive way of simply iterating through objects against other objects in two nested loops did not work, as the loops would need to be implemented with two systems in the ECS sense, and those systems could not be nested. Since this was not possible, another approach was used: at the beginning of the frame, a system was run for all objects in the scene that would be collided against, and for each object, a bit array was updated to reflect that their position is occupied — each bit representing one coordinate in the world grid. Later, when applying movement, the bit representing the new position's coordinate would be checked to see if the particular tile was already occupied. This method likely performed much better due to effectively reducing the time complexity of the movement logic from $O(n^2)$ to $O(n)$, but there might be other similar cases where is no workaround, or the workaround is less preferable.

As the prototype game engine did not implement text rendering, yet the game needed to communicate some game state to the player which could not be easily shown via iconography, the game ended up having many image resources containing text, in a way "pre-rendering" the text, as seen in Figure 5.2. Game engines often include a text renderer, so that games can display dynamically formed text. The most dynamic state in the game that would have benefited from a text renderer was the characters' oxygen and morale levels, which could have been displayed with numbers, but were instead rendered using tally marks: the game contained five sprites for representing a tally from one to five, and

Figure 5.2: A closer screenshot of the character status pages in the game. Note how none of the text is rendered with a consistent font, rather being hand-drawn since all the text had to be drawn into the image itself anyway, and how the countable details about the character are represented with tally marks, rather than arabic numerals.

multiple of these sprites could be drawn next to each other to represent numbers higher than five — in retrospect, it probably would not have been much harder to have sprites for ten digits, and handle their layout on the game side, much like the tally marks. Still, a text renderer would have made this even easier, and would not have required drawing text into the game's sprites directly.

There was one last-minute feature addition to the engine during the development of the game, namely support for Emscripten builds. Emscripten is a toolchain [6] for compiling regular programs into WebAssembly and JavaScript, to allow embedding said programs in a web page — a useful capability in a game jam, where many reviewers prefer testing games that are playable in their browser. Since SDL — the library used for platform-specific functionality in the engine — supports Emscripten as a target, the porting was simple, but needed a few tweaks nonetheless: high-resolution rendering for high-DPI displays needed to be disabled due to a rendering bug when it was enabled, and the multithreaded file reading implementation needed to be replaced for web builds.

In the end, there was one subsystem that had a major issue, the platform-independent part of audio playback. The system is relatively complex due to having to translate between many offsets: one to translate from "audio time" to a specific offset into the audio buffer, another to pick the appropriate chunk from the resource manager, and a third to index into that chunk. Due to a lack of testing, these offset calculations were wrong when playing back audio clips stored in many chunks — the audio clips used for testing this feature

happened to be short enough to fit within a single chunk.

Taking into account the aforementioned issues, the prototype game engine did still fulfill its purpose, and the game was playable from an early state, all the way up to release, where no major issues were reported by the 21 reviewers — other participants who tested and gave feedback on the game.

5.4 Evaluation

As seen in Chapter 4, Objectives O1, O2, and O3 were the main focus in the design phase, and a large part of the development process revolved around implementing these feature requirements. Objective O4 took less design and development time, but ended up being implemented in a similar manner to ggez mentioned in Section 2.4.2. The rest of this section covers the degree to which these objectives were achieved in more detail.

It is safe to say that the artifact satisfies Objective O1, *"a custom memory allocator for per-frame data, on both the engine and game sides."* Most of the allocation needs of the engine ended up being fulfilled by the linear allocator, which works best when used for per-frame data, as the memory can both be reused, and lifetimes are not an issue, as explained in Observation 2 in Section 5.1. Additionally, both the persistent linear allocator used to allocate memory for the static lifetime, and the per-frame linear allocator used to allocate memory for the length of a single frame, were used in the game described in Section 5.3, so the per-frame allocator also works for game code.

Objective O2, *"a custom memory allocator for game assets, for the engine's resource manager,"* is less clear cut, as the final prototype did not end up using an allocator, at least anything named as such, for resource data. That said, dynamically loading relevant chunks in the sparse array containing the resource data, and reusing the memory behind those chunks as needed, does result in a similar usage pattern to having those chunks be allocated from a pool allocator. Thus the design ended up working in practice, even though it was not implemented via an explicit allocator.

Objective O3, *"memory-safe utilities for multithreading, for both the engine and game code,"* was achieved in the sense that the engine does have a safe multithreading API, the `parallelize` function mentioned in Section 4.2, which is usable both on the engine and game side. This functionality was used internally in the engine by the audio mixer, but the demonstration game did not end up making use of it — though not because it would

have been problematic in any way, but because there were no performance problems that would have motivated it.

Finally, Objective O4, *"engine systems should be easily usable from other systems as well as game code,"* was at least achieved in the sense that there were no difficulties during the development of neither the engine, nor the demonstration game, regarding accessing different subsystems of the engine. Generally, everything that needed access to multiple subsystems also had a mutable borrow of the `Engine` type, which contains all of the subsystems of the engine, which could then also be used mutably. However, this likely worked as well as it did due to the simplicity of both the engine and the game made with it, as both of them could implement most functionality that needed access to multiple subsystems within a single function that has the mutable `Engine` borrow. This is significant, since the borrow checker allows for more granular borrowing patterns when all of the code is in a single function: for example, you could immutably borrow and share references to the resource database — which is a field inside the `Engine` structure — and then mutably still access the audio mixer — another field in the `Engine` structure — without the borrow checker complaining that a struct from the first section still holds shared references to the resource database. This does not work if you had two separate functions accessing those fields, as a function that takes a mutable borrow of `Engine`, and returns a struct containing a borrow of the resource database, would end up preventing any further borrows of `Engine` due to the mutable borrow passed into the function being bound to the lifetime of the borrow of the resource database inside the returned struct. This could be circumvented by passing a borrow of the resource database directly to said function instead of passing in the `Engine` borrow, but this can get annoying from the engine user's perspective, as each subsystem needs to be explicitly passed into every function that eventually calls another function that needs to access said subsystems.

6 Discussion

This chapter considers the results from the perspective of the research questions set out in Chapter 1, discusses the limitations of this study, and concludes the thesis with an overview of the findings and potential avenues for future work around this topic.

6.1 Research Questions

RQ1: "Which parts of game engine development are hindered by using Rust?"

Based on the design and development process and the observations made, using the Rust programming language specifically does seem to constrain the implementation of the *Core Systems* of a game engine, compared to implementing one in C++. Planning how to allocate memory in a way that can be represented in Rust lifetimes, which allocations should have dynamic lifetimes and how are those dynamic lifetimes handled safely, and how should memory for asynchronous tasks be owned, are all design questions that arise from the borrow checker. Even though many of these could be useful to think about in the case of other languages, it is not mandatory in those languages, as they handle the relevant safety issues differently, and make different assumptions about the safety of surrounding code. While Rust does help make safe code, it does also assume that all safe code really is safe.

RQ2: Which game engine design patterns fit Rust well?

While constraining the development process, using Rust did also lead to some elegant solutions for some particular problems. The per-frame allocator fits Rust well, as the lifetime can be represented perfectly for the borrow checker, which leads to the concept not really having many downsides — not only is it performant, but with the borrow checker, it is memory safe as well. The scatter-gather pattern of multithreading also works well for a similar reason, because the pattern can be exposed with a function that ensures — at compile time — that memory is accessed in a safe way for a multithreaded process: memory that needs to be written to is split between threads to avoid the need for synchronization, and memory that only needs to be read, can be read by all threads.

RQ3: How do Rust's language features affect game engine API design?

The prototype game engine ended up being similar to ggez, storing the game engine state in one big struct with subsystems accessible from that, which does appear to be the most ergonomic way to do it in Rust. This method does also appear to be workable at least for small games, as mentioned in the evaluation of Objective O4 in Section 5.4. For larger games, the approach taken by Bevy, having a more explicitly managed method of accessing various parts of the engine for various purposes, does seem like a better fit, but was not evaluated by the prototype that was made due to the complexity of implementing such a system. In the end, it appears to be hard to approach the simplicity of the traditional singleton method seen in C++ game engines — though that might come with its own problems, since as noted before regarding RQ1, restrictions imposed by Rust do address real safety concerns.

6.2 Limitations

Due to the complex nature of game engines, studying their development does come with some major limitations to both the validity and generalizability of these findings. To begin with, no two game engines are the same, and another developer writing a game engine in Rust would most likely have made a very different piece of software. That said, due to both pieces of software having the same use case and thus requirements, it seems likely that there would be at least some overlap in the more general observations of the development process, such as those discussed in Section 5.1.

The scope of both the prototype game engine, and the game written to demonstrate its usage, do limit the usefulness of the findings when it comes to more extensive engines for large and complex game projects — even a project of this size ended up coming across surprises, such as the difficulty in implementing a pool allocator, but also the usefulness of the linear allocator which can only allocate memory with an explicit lifetime. Though again, some of the observations were both high-level enough, and particular to patterns found all across game engines, rather than some specific system, that they may be more widely applicable.

Another difference the prototype game engine might have with a more complex project is the standard library usage: if another game engine were to use the Rust standard library's allocation, multithreading, and file reading utilities, they could not only sidestep a large majority of the difficulties encountered during this project related to those systems' implementation, but they could end up with entirely different memory allocation patterns due

to how ergonomic the allocating functions in the standard library are — as a prime example, the `Vec` dynamic-length array type allows writing code that does not pre-allocate its memory, rather allocating and reallocating more memory as needed, which is a completely different approach than what the prototype game engine took.

Finally, the resource database implemented for the prototype game engine avoids a whole class of complexity related to dependencies between resources, which likely made the streaming system much simpler to implement than a game engine with more complex requirements would need, and could potentially have had large implications on the design of the resource database. In particular, any game engine including a 3D renderer would need to take these resource dependencies into account, due to the complex reuse of triangle mesh data, textures, material properties, and shaders between separate resources.

6.3 Conclusion

This thesis has shown that Rust seems to be a viable language for game engine development, at least for small games. The safety features of the language evidently helped the development process, as even though this project included low-level systems including custom memory allocators and multithreading utilities, no memory corruption, use-after-frees, or unchecked out-of-bounds accesses were actually observed. On the other hand, instead of dealing with such bugs, the development process did require a lot of design work to make the prototype game engine work with Rust's requirements for clear and compile-time validatable lifetimes, as well as quite a bit of manual validation work for unsafe code, particularly in the core systems.

Regarding Rust's language features for game engines specifically, the borrow system and lifetimes fit the needs of game engines particularly well: all frame-temporary data can be safely allocated for just the one frame, and much of the persistent memory can be allocated up front, which can be modeled well with static borrows. Indirect dispatch being clearly communicated through the `dyn` keyword allowed both avoiding it for performance most of the time, but also specifically making use of it where it matters and does not cause performance issues: for the platform layer, which allowed parallelizing the compilation of the two major halves of the project. One feature that could have made this prototype better is user-provided allocators for the standard library's allocating data structures: while the global allocator can be changed, picking a specific allocator for a particular object is currently not possible with the Rust standard library without unstable features

— though this feature becoming stabilized seems to only be a matter of time, as there is a dedicated Rust Allocators Working Group [20] working on this topic.

Future studies around this topic could compare how using the rest of the Rust standard library — the `std` crate — would affect the development process. Based on the observations from this thesis, it seems likely to speed up development, though runtime performance problems could arise in turn. Another topic not explored in this thesis with great potential is Rust's async/await syntax, which could be used to great effect in many game engine subsystems, if an asynchronous executor was implemented specifically for the engine. GPU compute, rendering, and transfer tasks could be expressible with the async/await syntax, and the executor could make use of fences to facilitate this behind the scenes. The Rust ecosystem already has good solutions for asynchronous file and network I/O, which games could make use of as well. Finally, gameplay mechanics might also benefit from asynchronous processes, perhaps using some manual polling of tasks, once per frame, as there is no real underlying asynchronous process, but having the ability to represent long-lived sequences of gameplay effects as asynchronous functions and processing them concurrently could be useful.

Bibliography

[1] C. Anderson and other contributors. *Bevy Engine*. https://bevyengine.org/. Accessed: 2024-09-02. 2020.

[2] E. F. Anderson, S. Engel, P. Comninos, and L. McLoughlin. "The case for research in game engine architecture". In: *Proceedings of the 2008 Conference on Future Play: Research, Play, Share*. Future Play '08. Toronto, Ontario, Canada: Association for Computing Machinery, 2008, pp. 228–231. ISBN: 9781605582184. DOI: 10.1145/1496984.1497031.

[3] A. Ben-Yehuda. *std::thread::JoinGuard (and scoped) are unsound because of reference cycles*. https://github.com/rust-lang/rust/issues/24292. Accessed: 2025-05-14. Apr. 2015.

[4] J. Carmack. *Static Code Analysis*. https://web.archive.org/web/20140713032309/http://www.altdev.co/2011/12/24/static-code-analysis/. Accessed: 2024-09-02. Dec. 2011.

[5] J. Clay and other contributors. *Tetra (game engine)*. https://github.com/17cupsofcoffee/tetra. Accessed: 2024-09-02. 2018.

[6] Emscripten authors. *Emscripten, a complete compiler toolchain to WebAssembly, using LLVM*. https://emscripten.org/. Accessed: 2025-04-14. 2012.

[7] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Seventeenth Symposium on Usable Privacy and Security (SOUPS 2021)*. USENIX Association, Aug. 2021, pp. 597–616. ISBN: 978-1-939133-25-0. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[8] ggez contributors. *ggez (game engine)*. https://ggez.rs/. Accessed: 2024-09-02. 2016.

[9] J. Gregory. *Game Engine Architecture, 3rd Edition*. CRC Press, 2019. ISBN: 978-1-4987-8365-1.

[10] C. Gyrling. *Are we out of memory?* https://www.swedishcoding.com/2008/08/31/are-we-out-of-memory/. Accessed: 2025-04-14. Aug. 2008.

[11] R. Jung, H.-H. Dang, J. Kang, and D. Dreyer. "Stacked borrows: an aliasing model for Rust". In: *Proc. ACM Program. Lang.* 4.POPL (Dec. 2019). DOI: 10.1145/3371109.

[12] Kalderon, Eyal and Amethyst contributors. *Amethyst (game engine)*. https://github.com/amethyst/amethyst. Accessed: 2024-09-02. 2016.

[13] S. Klabnik and C. Nichols. *The Rust Programming Language*. https://doc.rust-lang.org/book/. Accessed: 2024-11-13. 2022.

[14] LogLog Games. *Leaving Rust gamedev after 3 years*. https://loglog.games/blog/leaving-rust-gamedev/. Accessed: 2024-09-02. Apr. 2024.

[15] MSRC Team. *A proactive approach to more secure code*. https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/. Accessed: 2024-11-17. 2019.

[16] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. "A Design Science Research Methodology for Information Systems Research". In: *Journal of Management Information Systems* 24.3 (2008), pp. 45–77. DOI: 10.2753/MIS0742-1222240302.

[17] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding memory and thread safety practices and issues in real-world Rust programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI 2020. New York, NY, USA: Association for Computing Machinery, 2020, pp. 763–779. ISBN: 978-1-4503-7613-6. DOI: 10.1145/3385412.3386036.

[18] S. Szpakowski and other contributors. *LÖVE (game engine)*. https://love2d.org/. Accessed: 2024-11-13. 2010.

[19] The Miri Team. *Miri, an interpreter for Rust's mid-level intermediate representation*. https://github.com/rust-lang/miri. Accessed: 2025-04-14. 2015.

[20] The Rust Project Contributors. *Rust Allocators Working Group*. https://github.com/rust-lang/wg-allocators. Accessed: 2025-06-18. 2019.

[21] The Rust Project Contributors. *The Rust Core Library*. https://doc.rust-lang.org/core/index.html. Accessed: 2025-04-02. 2016.

[22] The Rust Survey Team. *2023 Annual Rust Survey Results*. https://blog.rust-lang.org/2024/02/19/2023-Rust-Annual-Survey-2023-results.html. Accessed: 2024-10-23. Feb. 2024.

[23] M. Toftedahl and H. Engström. "A Taxonomy of Game Engines and the Tools that Drive the Industry". In: *Proceedings of DiGRA 2019 Conference: Game, Play and the Emerging Ludo-Mix*. Tampere: DiGRA, 2019. DOI: 10.26503/dl.v2019i1.1077.

[24] N. Villani. *Tree Borrows*. https://perso.crans.org/vanille/treebor/. Accessed: 2025-05-14. Mar. 2023.

[25] D. Wood. *Polymorphisation: Improving Rust compilation times through intelligent monomorphisation*. https://web.archive.org/web/20241203080519/https://davidtw.co/media/masters_dissertation.pdf. Accessed: 2025-06-24. MA thesis. Apr. 2020.

[26] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ASE '22. New York, NY, USA: Association for Computing Machinery, 2023. ISBN: 978-1-4503-9475-8. DOI: 10.1145/3551349.3559494.

[27] S. Zhu, Z. Zhang, B. Qin, A. Xiong, and L. Song. "Learning and programming challenges of rust: a mixed-methods study". In: *Proceedings of the 44th International Conference on Software Engineering*. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 1269–1281. ISBN: 978-1-4503-9221-1. DOI: 10.1145/3510003.3510164.

Appendix A Development time table

The following table contains the data tracked during the development process described in Section 4.2. During development, each individual task for implementing a particular feature was tracked for time, the section of the engine — as described by Figure 2.1 — what kind of task it was, and which Rust-specific topics were relevant during that task.

In the original task tracking document, tasks could be marked as developing multiple engine parts in one task — for easier analysis and to make the table easier to fit, these were split into multiple rows in this table, listing each engine part as its own row, with the time split evenly between the different engine parts. For example, the first two rows describe one task that took 2 hours and 40 minutes to finish.

The third, fourth, fifth and sixth columns describe what kind of task it was, in order: feature development, bug fixes, code cleanup, and testing.

The seventh, eighth, ninth, and tenth columns describe the Rust-related topics that were relevant to the task, in order: refactoring, lifetimes, ownership, and unsafe code.

Time	**Engine part**	**Fe.**	**Bu.**	**Cl.**	**Te.**	**Re.**	**Li.**	**Ow.**	**Un.**
1h 20m	3rd Party SDKs	✓						✓	
1h 20m	Platform Independence Layer	✓						✓	
1h 30m	Core Systems	✓						✓	
0h 13m	Platform Independence Layer			✓		✓			
4h 8m	Core Systems	✓							✓
4h 28m	Core Systems	✓						✓	✓
0h 17m	Core Systems			✓		✓			
1h 31m	Core Systems				✓				
0h 27m	Core Systems		✓		✓				
4h 41m	Platform Independence Layer	✓						✓	
1h 26m	HID	✓							
2h 10m	HID	✓							
2h 11m	Platform Independence Layer	✓							
10h 42m	Core Systems	✓					✓	✓	
27h 5m	Resources (Game Assets)	✓					✓	✓	
15h 49m	Resources (Game Assets)	✓	✓	✓		✓	✓		
10h 40m	Resources (Game Assets)	✓					✓		
6h 41m	Platform Independence Layer						✓		
1h 24m	Platform Independence Layer			✓		✓			

7h 24m	Core Systems	✓				✓			✓
14h 44m	Platform Independence Layer	✓					✓		✓
6h 8m	Core Systems	✓				✓			✓
8h 0m	Resources (Game Assets)	✓							
7h 56m	Low-Level Renderer	✓							
14h 17m	Asset Conditioning Pipeline	✓	✓						
3h 2m	Low-Level Renderer			✓		✓			
8h 0m	Resources (Game Assets)	✓		✓			✓		
7h 3m	Audio	✓		✓			✓		
3h 0m	Core Systems	✓		✓		✓	✓		✓
3h 0m	Resources (Game Assets)	✓	✓	✓		✓			
2h 32m	Audio		✓	✓					
2h 9m	Gameplay Foundations								
25h 51m	Gameplay Foundations	✓			✓				
2h 11m	Profiling & Debugging	✓							
6h 13m	Core Systems	✓					✓	✓	
5h 13m	Platform Independence Layer		✓			✓			✓
0h 20m	Resources (Game Assets)	✓				✓			
0h 44m	Platform Independence Layer			✓					
0h 2m	Core Systems				✓				
2h 18m	Core Systems		✓			✓	✓		
1h 26m	Platform Independence Layer			✓					
0h 38m	Core Systems		✓						
4h 16m	Core Systems		✓			✓			✓
1h 38m	Gameplay Foundations			✓		✓			